

Université Claude Bernard Lyon 1

IUT DOUA Dpt : Informatique

43 BV 11 Novembre 1918

69100 Villeurbanne

Appy Makers

Service Informatique

22 rue Marseille

69007 Lyon

RAPPORT DE STAGE

Wa'hil BOUDRAA

*BUT INFO 2A parcours Réalisation d'Applications : conception,
développement, validation*

**Développeur No Code / Low Code : Conception et
développement d'applications métiers sur mesure**

Du 07/04/2026 au 26/06/2026 (Année 2026)

TUTEUR ENSEIGNANT

Mme Mariette BESSAC

IUT DOUA - Informatique

MAÎTRE DE STAGE

M. Frédéric LACROIX

Dirigeant Appy Makers

Remerciements

La réalisation de ce stage et la rédaction de ce rapport ont été rendues possibles grâce à l'accompagnement de plusieurs personnes que je tiens à remercier chaleureusement.

J'exprime tout d'abord ma profonde gratitude à **M. Frédéric Lacroix** et **M. Vincent Stivala**, dirigeants de l'entreprise AppyMakers, pour m'avoir accueilli au sein de leur structure. L'opportunité qu'ils m'ont offerte de travailler sur des projets concrets a été une véritable source de motivation.

Je tiens à adresser des remerciements tout particuliers à **M. Frédéric Lacroix**, mon maître de stage. Sa bienveillance, sa grande patience et son écoute ont grandement facilité mon intégration au sein de l'entreprise. Je le remercie sincèrement pour le temps qu'il m'a accordé au quotidien et pour la richesse des connaissances techniques qu'il m'a transmises, m'aidant ainsi à monter en compétence tout au long de cette expérience.

Je remercie sincèrement **Mme Mariette Bessac**, ma tutrice de stage. Sa réactivité lors de la signature de ma convention de stage m'a permis d'aborder cette première expérience en entreprise sereinement. Je la remercie également pour son accompagnement global, ainsi que pour sa visite dans nos locaux, les conseils et les informations qu'elle m'a donnés à cette occasion m'ont été d'une aide précieuse.

Ma reconnaissance s'adresse également à l'ensemble de l'équipe technique d'AppyMakers : **Omar Elhadidi**, **Fernando Sandoval Egli**, **Arthur Yvars** et **Quentin Trotobas**. Je les remercie pour leur accueil chaleureux, leur disponibilité et leur formidable esprit d'équipe.

Enfin, je tiens à remercier **Mme Jocelyne Debouté** d'avoir partagé les offres de stage au sein de notre département. Sans cette initiative, je n'aurais probablement pas eu l'opportunité d'effectuer mon stage dans cette merveilleuse entreprise.

Fiche Technique du Stage

Cette fiche synthétise les caractéristiques principales et l'environnement technique de mon stage chez AppyMakers.

Rubrique	Informations
L'ÉTUDIANT	Nom et Prénom : BOUDRAA Wa'hil Téléphone : 06 68 67 79 20
ORGANISME D'ACCUEIL	Raison Sociale : Appy Makers Secteur : Conseil en SI et développement d'application Adresse : 12 rue Saint Polycarpe, 69001 Lyon Lieu du stage : 22 rue de Marseille, 69007 Lyon
ENCADREMENT	Maître de stage : M. Frédéric LACROIX (Dirigeant) Contact : frederic.lacroix@appymakers.com / 06 68 91 81 52
MODALITÉS DU STAGE	Période : Du 7 avril 2026 au 26 juin 2026 Volume Horaire : 13 semaines (392 heures) Poste : Développeur No Code – Conception et développement d'applications métiers sur mesure
MISSIONS CONFIÉES	Refonte intégrale du portail ConcessLink pour le réseau Fassi France. Développement d'écrans métiers et de tableaux de bord (Front-end). Modélisation de base de données et intégration d'APIs (Back-end). Conception de composants UI dynamiques réutilisables (No-Code et Vue.js).
TECHNOLOGIES	Front-end : WeWeb, Vue.js Back-end / API : Xano IA Générative : Claude, Gemini, etc.
DÉROULEMENT	Phase 1 : Découverte des outils No Code et observation ; Phase 2 : Participation active aux projets clients ; Phase 3 : Bilan et consolidation des acquis en autonomie

SOMMAIRE

Remerciements.....	1
Fiche Technique du Stage.....	2
Introduction Générale.....	4
I. Présentation de l'entreprise et de l'environnement de travail.....	5
I.1. L'entreprise AppyMakers et son secteur d'activité.....	5
I.1.a. Historique et positionnement.....	5
I.1.b. Une organisation autour de deux pôles d'expertise.....	5
I.1.c. Le choix stratégique du Low-Code / No-Code.....	6
I.2.a. Une équipe à taille humaine.....	6
I.2.b. Les conditions de réalisation.....	7
I.3. Outils de travail, méthodologie et prise en main.....	8
I.3.a. Les outils de l'écosystème AppyMakers.....	8
I.3.b. Méthodologie et gestion de projet.....	8
I.3.c. Ma phase d'intégration technique.....	9
II. La conception et le développement des écrans métiers.....	9
II.1. Le projet ConcessLink : un portail B2B pour le réseau Fassi France.....	9
Un schéma de travail récurrent.....	10
II.2. L'espace Marketing & Commerce.....	10
II.2.a. L'écran Brochures : mon premier écran complet.....	10
II.2.b. Les écrans Supports salon et Goodies.....	12
II.2.c. La Médiathèque : gérer fichiers et mots-clés.....	14
II.2.d. Les Documents commerciaux.....	15
II.3. L'espace Administration.....	16
II.4. L'espace Service Après-Vente.....	18
II.7. État d'avancement global du projet.....	20
III. La réalisation de composants réutilisables et de logique back-end.....	20
III.1. Le composant Active Filter Chips (no-code).....	20
III.2. Le composant Input Number (codé en Vue.js).....	21
Le cheminement technique.....	21
Le cœur de la logique.....	22
III.3. Le composant Counter.....	23
III.4. La fonction back-end create_missing_tags (Xano).....	24
IV. Bilan des compétences mises en œuvre et acquises.....	25
IV.1. La gestion de projet et l'organisation des missions.....	25
IV.2. Compétence « Réaliser » (UE1) : du code écrit à la main à l'application complète.....	27
IV.3. Compétence « Optimiser » (UE2) : appliquer l'algorithmique en contexte réel.....	27
IV.4. Compétence « Gérer » (UE4) : la modélisation et l'exploitation des données.....	28
IV.5. Les autres compétences : Administrer, Conduire, Collaborer.....	29
IV.6. Synthèse auto-évaluative.....	29
Conclusion générale.....	30

Introduction Générale

Le diplôme de Bachelor Universitaire de Technologie (BUT) en Informatique, et plus particulièrement le parcours Réalisation d'Applications (RA), prépare les étudiants à concevoir, développer et maintenir des solutions logicielles professionnelles. À la fin de la deuxième année d'études, un stage de 8 semaines minimum est obligatoire. Ce stage représente notre premier contact concret avec le monde de l'entreprise. Il nous permet de mettre en pratique les connaissances théoriques acquises à l'IUT Lyon 1 (en algorithmique, conception de bases de données et développement web) tout en développant notre savoir-être professionnel.

C'est dans ce contexte que j'ai effectué mon stage, du 7 avril au 26 juin 2026, au sein de la société AppyMakers. Située à Lyon, cette jeune agence de services numériques (ESN) s'est spécialisée dans la transition digitale des entreprises à l'aide des outils dits "Low-Code" et "No-Code". Ces technologies, en pleine expansion, permettent de concevoir des applications web et mobiles sur mesure beaucoup plus rapidement que le développement traditionnel, tout en conservant une vraie exigence technique.

J'ai occupé durant ce stage le poste de Développeur No-Code. Pour un étudiant habitué à coder "de zéro" (en Java, PHP ou JS), ce poste soulève une question intéressante : comment nos cours de programmation s'appliquent-ils dans un environnement de développement visuel ? J'ai rapidement compris que, même si l'on n'écrit pas toutes les lignes de code manuellement, la logique sous-jacente reste strictement identique. Développer avec des outils comme Xano et WeWeb demande de solides notions d'architecture, une bonne modélisation de bases de données relationnelles et une maîtrise des appels d'APIs.

Mes attentes pour ce stage étaient claires : je souhaitais participer activement à la création d'applications concrètes, découvrir le fonctionnement d'une équipe de développement agile et apprendre à utiliser de nouvelles technologies pour enrichir mes compétences. J'ai eu l'opportunité de travailler sur l'application ConcessLink, en intervenant aussi bien sur le back-end que sur le front-end.

Afin de rendre compte de cette expérience enrichissante, ce rapport est structuré en trois grandes parties : La première partie présentera l'entreprise AppyMakers, son secteur d'activité ainsi que mon environnement de travail au quotidien. La deuxième partie sera consacrée à l'explication méthodique de mes missions et de mes réalisations techniques. Enfin, la troisième partie proposera un bilan structuré des compétences acquises, en lien avec le référentiel de notre formation.

I. Présentation de l'entreprise et de l'environnement de travail

I.1. L'entreprise AppyMakers et son secteur d'activité

I.1.a. Historique et positionnement

AppyMakers est une entreprise lyonnaise fondée très récemment, en juillet 2023, par **Frédéric Lacroix** et **Vincent Stivala**. Il s'agit d'une Société par Actions Simplifiée (SAS) dotée d'un capital social de 100 000 €. Son siège social est situé au 22 rue de Marseille, dans le 7ème arrondissement de Lyon.

L'entreprise se positionne comme une agence de conseil et de services numériques (ESN). Son cœur de métier est l'accompagnement des Petites et Moyennes Entreprises (PME) ainsi que des Entreprises de Taille Intermédiaire (ETI) dans leur transformation numérique. AppyMakers crée pour ces clients des outils internes sur mesure : des ERP (progiciels de gestion intégrés), des CRM (gestion de la relation client), des tableaux de bord ou encore des applications métiers spécifiques.

I.1.b. Une organisation autour de deux pôles d'expertise

Pour mener à bien cette stratégie de digitalisation complète, l'activité d'AppyMakers s'articule autour de deux pôles d'expertise distincts mais parfaitement complémentaires :

- **Le pôle Conseil en MOA (Assistance à Maîtrise d'Ouvrage)** : Avant même d'envisager la création technique d'une application, ce pôle intervient en amont pour accompagner le client. Son rôle est d'analyser et de modéliser les processus métiers de l'entreprise cliente, de définir avec précision le besoin, de rédiger les spécifications fonctionnelles (le cahier des charges) et de conseiller la solution technologique la plus pertinente. Ce pôle fait office de traducteur indispensable entre le "langage métier" du client (la Maîtrise d'Ouvrage) et l'équipe de développement.

- **Le pôle Digital Factory** : C'est le pôle de réalisation technique et d'intégration, auquel j'étais rattaché durant mon stage. La Digital Factory (ou "usine numérique") prend le relais de la MOA pour concevoir, développer et tester les solutions logicielles. En s'appuyant sur l'agilité des outils Low-Code, cette équipe fonctionne par itérations rapides (livraisons fréquentes de fonctionnalités) pour s'assurer que le produit technique correspond exactement aux attentes définies par le pôle Conseil.

Cette double casquette permet à AppyMakers de ne pas être un simple exécutant technique, mais un véritable partenaire stratégique pour ses clients, ce qui fait sa force sur le marché.

I.1.c. Le choix stratégique du Low-Code / No-Code

La particularité d'AppyMakers sur le marché des ESN réside dans son utilisation exclusive des technologies Low-Code et No-Code (Xano et WeWeb). Sur le marché actuel, de nombreuses entreprises ont besoin de se digitaliser mais n'ont ni le budget ni le temps d'attendre les mois de développement que nécessite une approche classique.

Les outils Low-Code permettent de concevoir des applications robustes de manière beaucoup plus agile. L'approche d'AppyMakers se déroule généralement en cinq grandes étapes pour chaque client : la compréhension fine du besoin métier, l'exploration des solutions possibles, le choix de la "stack" d'outils la plus adaptée, la phase de mise en œuvre technique (développement), et enfin l'accompagnement sur le long terme.

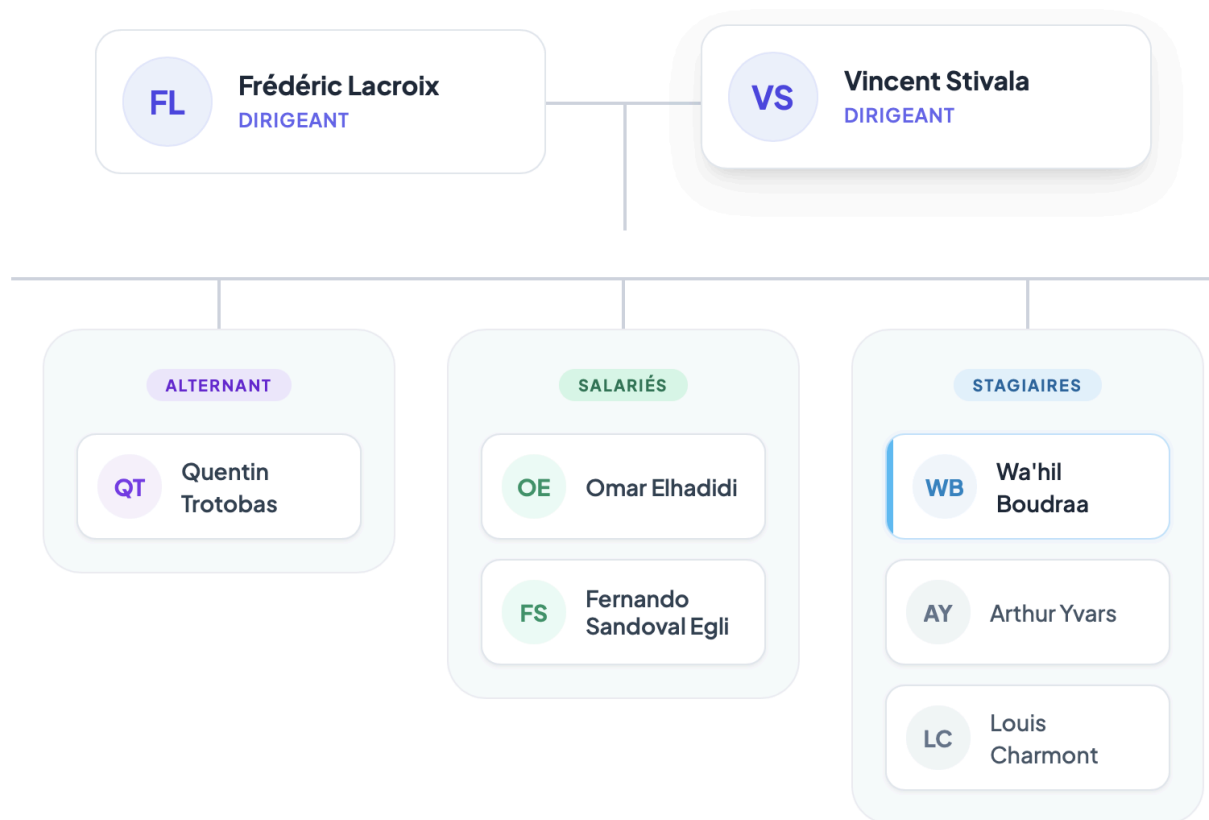
I.2. L'environnement de stage : l'équipe et les conditions de réalisation

I.2.a. Une équipe à taille humaine

L'un des critères qui m'a poussé à choisir AppyMakers était la taille de l'entreprise. Travailler dans une petite structure de six personnes favorise une intégration rapide et permet d'avoir une vision globale des projets, contrairement à de grandes entreprises où les tâches sont souvent très segmentées.

L'organigramme de l'entreprise est très horizontal, ce qui facilite grandement la communication :

- **La direction** : **Frédéric Lacroix** (mon maître de stage principal) et **Vincent Stivala**, qui gèrent l'entreprise, la relation client et la direction de projet.
- **Le pôle développement** : composé de **Quentin Trotobas**, **Arthur Yvars**, **Omar Elhadidi** et **Fernando Sandoval Egli**.



L'ambiance de travail s'est révélée très conviviale et presque familiale. Cette proximité hiérarchique m'a permis d'échanger naturellement avec tout le monde, de poser des questions sans retenue et d'obtenir des retours techniques rapides, ce qui est particulièrement formateur lors d'un premier stage.

I.2.b. Les conditions de réalisation

Le stage s'est déroulé sur une base de 35 heures hebdomadaires. Les locaux, situés à La Guillotière, offrent un espace de travail agréable. De plus, l'entreprise a mis en place un système adapté aux méthodes de travail modernes en informatique : l'équipe est en télétravail tous les jeudis. Cette organisation demande une bonne rigueur personnelle et une bonne maîtrise des outils de communication à distance, ce qui a représenté pour moi un excellent apprentissage de la vie professionnelle actuelle.

I.3. Outils de travail, méthodologie et prise en main

I.3.a. Les outils de l'écosystème AppyMakers

Pour mener à bien mes missions, j'ai dû me familiariser avec un ensemble d'outils professionnels, qui se divisent en deux catégories :

Outil de gestion de communication	
Outils	Description
	Microsoft Teams : utilisé pour la communication instantanée, les appels et les réunions.
	Notion : outil central de documentation pour les cahiers des charges, et suivi des projets (modèle kanban).

Outil de développement	
Outils	Description
	Xano : plateforme "Back-end as a Service" pour structurer la base de données (PostgreSQL) et développer les API.
	WeWeb : outil de création front-end pour concevoir des interfaces web interactives connectées aux données.
	Vue.js : framework JavaScript sur lequel repose WeWeb, utilisé pour coder sur mesure nos propres composants réutilisables.

I.3.b. Méthodologie et gestion de projet

AppyMakers ne suit pas des méthodes complexes comme Scrum, mais utilise plutôt une méthode Agile. Nos journées étaient rythmées par un "stand-up meeting" matinal (une réunion rapide d'équipe ou personnelle) où **M. Lacroix** distribuait ou ajustait les tâches de la journée. Un point informel était également fait l'après-midi pour suivre l'avancement. Bien que certaines tâches fussent documentées sur Notion (avec des colonnes "à faire", "en cours", "terminé"), une grande partie de la gestion des priorités se faisait à l'oral.

Cela m'a poussé à être très autonome dans ma propre organisation (notamment grâce à Notion pour la prise de note) pour ne perdre aucune consigne.



Backlogs Concess Link

Actif

Done

Archivés

Nouveau

Na	Name	Statut	Qui	Module	Priorité	Versions	Type
	Ecran Users	A valider	Wahil		Haute		
	Ecran Mon Profil	A faire	Frédéric		Haute		
	Header	A valider	Wahil		Haute		
	Gestion des Espaces	A valider	Wahil		Haute		
	Faire une fonction qui attribue des couleurs	A valider	Wahil		Moyenne		
	Uniformisation	A faire	Louis		Moyenne		
	Ecran Accueil SAV	A faire	Louis		Basse		
	Ecran Liste Tickets	A faire	Quentin		Basse		

I.3.c. Ma phase d'intégration technique

Avant ce stage, je n'avais jamais manipulé Xano ni WeWeb. Lors de ma première semaine, l'entreprise m'a laissé un temps d'autoformation précieux tout en me guidant. En m'appuyant sur la documentation officielle, j'ai développé une petite application "bac à sable" (gestionnaire de loueur de voitures) pour m'entraîner. Cette phase a été cruciale : elle m'a permis de comprendre comment lier une base de données Xano à un tableau WeWeb, et comment passer des requêtes réseau de manière sécurisée. Une fois cette logique assimilée, j'étais prêt à intervenir sur les vrais projets de l'entreprise.

II. La conception et le développement des écrans métiers

II.1. Le projet ConcessLink : un portail B2B pour le réseau Fassi France

Le client pour lequel j'ai travaillé est **Fassi France**, filiale française du constructeur italien Fassi Gru, leader sur le marché de la manutention embarquée et du levage (les grues articulées que l'on voit montées à l'arrière des camions). Le groupe regroupe aussi d'autres marques connues du secteur comme Marrel (bras de levage) ou Forez-Bennes. Fassi France s'appuie sur un réseau dense de plus de quatre-vingts concessionnaires et points d'accueil répartis sur tout le territoire.

Pour animer ce réseau, l'entreprise dispose d'un portail B2B un extranet réservé à ses concessionnaires et agents qui couvre tout le cycle de vie du produit, depuis l'avant-vente jusqu'au service après-vente. Ce portail est organisé en trois grands espaces : **Service Après-Vente**, **Avant-Vente** et **Marketing & Commerce**. C'est ce nouveau portail, appelé **ConcessLink**, qu'AppyMakers a été chargé de reconstruire et que j'ai contribué à développer.

Il faut préciser qu'un portail existait déjà. Lancé en 2021 et développé en quelques mois, il avait bien fonctionné au démarrage mais n'avait pas été dimensionné pour supporter la charge réelle d'utilisateurs : il provoquait de nombreux ralentissements. Une première tentative de refonte par un autre prestataire avait échoué, faute d'un périmètre correctement chiffré et d'un suivi rigoureux. L'enjeu pour AppyMakers était donc double : reconstruire un portail robuste et performant, tout en livrant des fonctionnalités régulièrement pour rassurer un client qui avait déjà été déçu.

C'est dans ce contexte que je suis intervenu, principalement sur **l'espace Marketing & Commerce**, qui est de loin celui sur lequel j'ai le plus travaillé, puis sur des écrans d'administration et quelques écrans de l'espace SAV en fin de stage.

Un schéma de travail récurrent

Avant de détailler chaque écran, je veux expliquer la logique commune à presque tous. Sur ConcessLink, un écran « métier » suit à peu près toujours le même schéma de réalisation, que j'ai pris l'habitude de dérouler :

D'abord le **back-end sur Xano** : je crée (ou je récupère) les tables nécessaires dans la base de données, puis je génère les API standards (lecture, ajout, modification, suppression ce que l'on appelle le CRUD) et, si besoin, des API spécifiques à l'écran avec une logique particulière. Ensuite le **front-end sur WeWeb** : je connecte une Collection (l'objet WeWeb qui exécute les requêtes API) à un tableau ou à des cartes, j'ajoute la recherche, les filtres et la pagination. Enfin un **back-office** réservé aux administrateurs, qui permet d'ajouter, de modifier et de supprimer les données via des fenêtres modales, le tout protégé par la gestion des droits que je décris plus loin. Beaucoup d'écrans comportent en plus un **panier** et un **formulaire de validation** qui déclenche l'envoi d'un e-mail de confirmation.

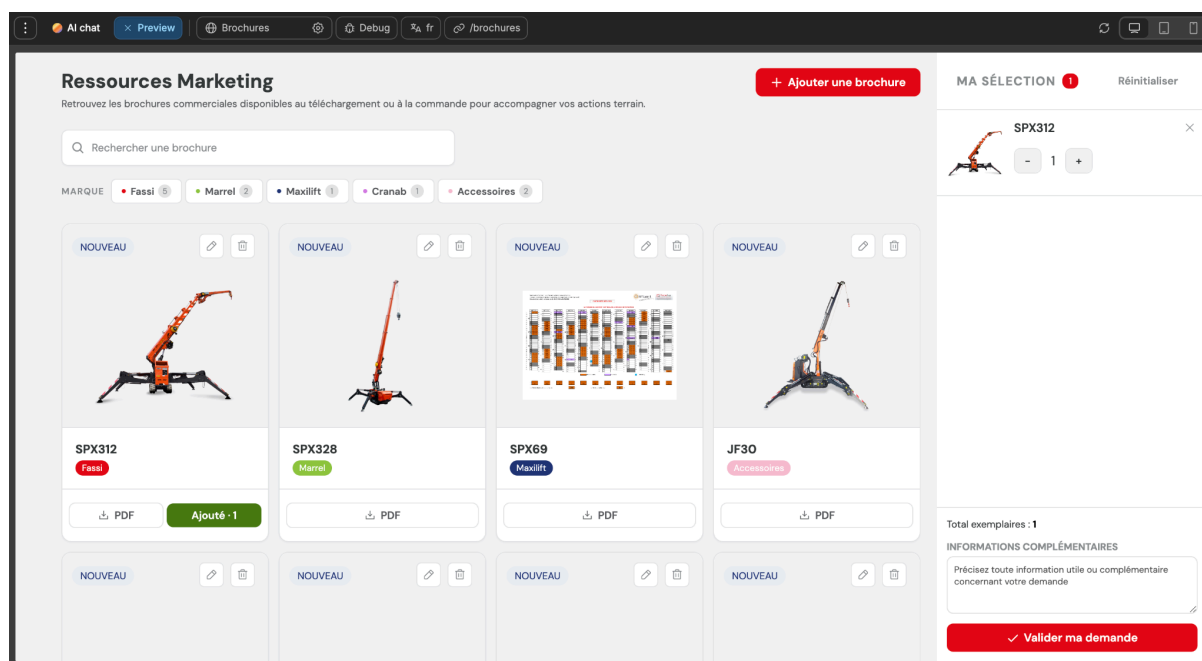
II.2. L'espace Marketing & Commerce

II.2.a. L'écran Brochures : mon premier écran complet

Les brochures ont été mon premier vrai écran sur le projet, et c'est sans doute celui qui m'a le plus appris. L'objectif était de permettre à un concessionnaire de consulter le catalogue des brochures de Fassi France, de les **rechercher**, de les **filtrer** par marque, de **télécharger** la version PDF, mais aussi de **commander** la version papier lorsqu'elle existe.

La commande passe par un panier : l'utilisateur ajoute des brochures, ajuste les quantités, puis valide une demande qui envoie automatiquement un e-mail aux gestionnaires de brochures ainsi qu'au client. Côté administrateur, le même écran devait permettre d'ajouter, de modifier et de supprimer une brochure.

Côté back-end, j'ai d'abord réalisé les API standards pour les tables `brochure` et `brochure_panier_ligne`. La recherche s'appuie sur un *fuzzy search* (une recherche « tolérante » qui retrouve un résultat même avec une faute de frappe ou un mot partiel), et le filtrage par catégorie se fait en passant à l'API GET un paramètre de type tableau d'entiers (`integer[]`). Pour l'affichage, j'ai disposé la sélection des brochures au centre et le panier sur la droite, sous forme de cartes.



Le développement du **panier** et de **l'envoi d'e-mail** est la première difficulté que j'ai rencontrée. Pour l'e-mail, j'ai créé une API dédiée et demandé à l'IA générative de me produire un modèle d'e-mail en HTML, que j'ai ensuite adapté pour qu'il respecte la charte de communication de l'entreprise. J'ai aussi dû gérer une limite métier : une ligne de panier ne peut pas dépasser une certaine quantité (vingt unités), avec un petit message d'avertissement au-delà.

Cet écran a également été le premier soumis à une **réunion de relecture avec la cliente**, et c'est là que j'ai compris une réalité du métier : un écran « qui fonctionne » n'est pas un écran « validé ». La cliente a demandé de nombreux ajustements réduire la taille des cartes, remonter les boutons d'administration, passer le bouton « Commander » en rouge, colorer les puces de filtre selon la marque (Fassi en rouge, Marrel en vert, Forez-Bennes en bleu), ouvrir le PDF dans un nouvel onglet plutôt que de le télécharger, ou encore remplacer un **toast** (petite notification éphémère) par une fenêtre modale pour la confirmation d'envoi.

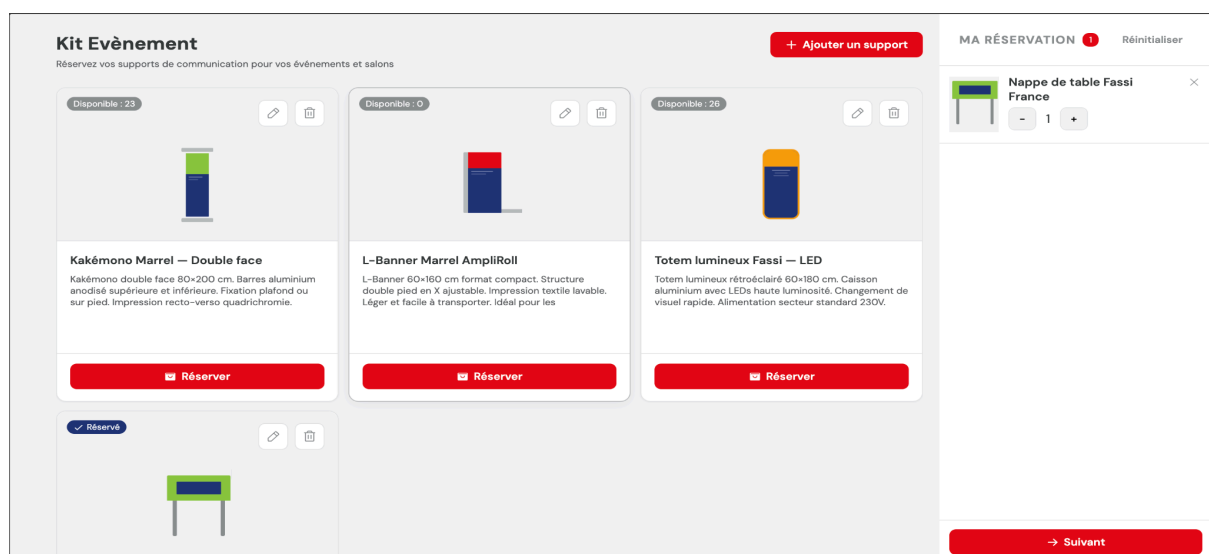
Reprendre tous ces points m'a appris à ne pas trop m'attacher à une première version et à concevoir des écrans plus faciles à faire évoluer.

État d'avancement : l'écran est terminé et fonctionnel ; il a servi de modèle pour les écrans suivants.

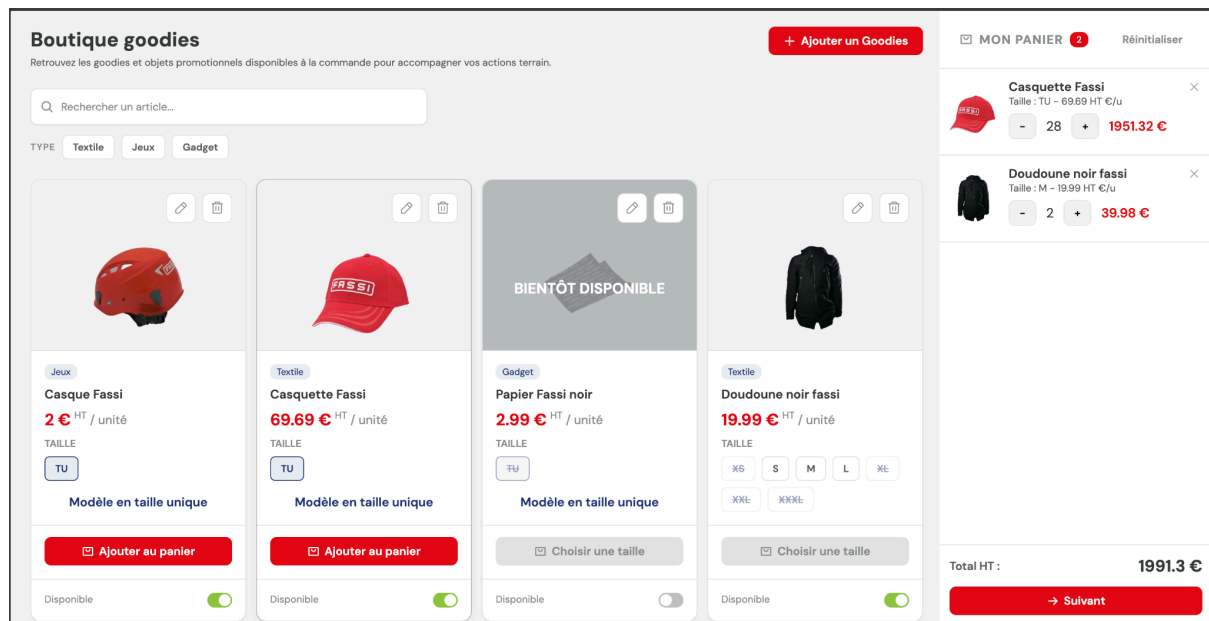
II.2.b. Les écrans Supports salon et Goodies

À partir de la semaine suivante, le rythme s'est accéléré : on m'a confié deux nouveaux écrans qui reposaient sur la même base que les brochures, ce qui m'a permis d'aller plus vite tout en consolidant mes acquis.

L'écran **Supports salon** présente le catalogue des supports utilisés sur les salons professionnels. On peut rechercher un support par son nom (sans filtre cette fois), l'ajouter au panier, puis valider en remplissant un formulaire qui déclenche un e-mail de confirmation. Le back-office permet là encore l'ajout, la modification et la suppression. Les tables `kit_salon` et `kit_salon_panier_ligne` existaient déjà. La principale difficulté technique a été de corriger une jointure (un `left join`) qui ne renvoyait pas les bonnes données, et de bloquer la quantité commandable à la quantité réellement disponible en stock.



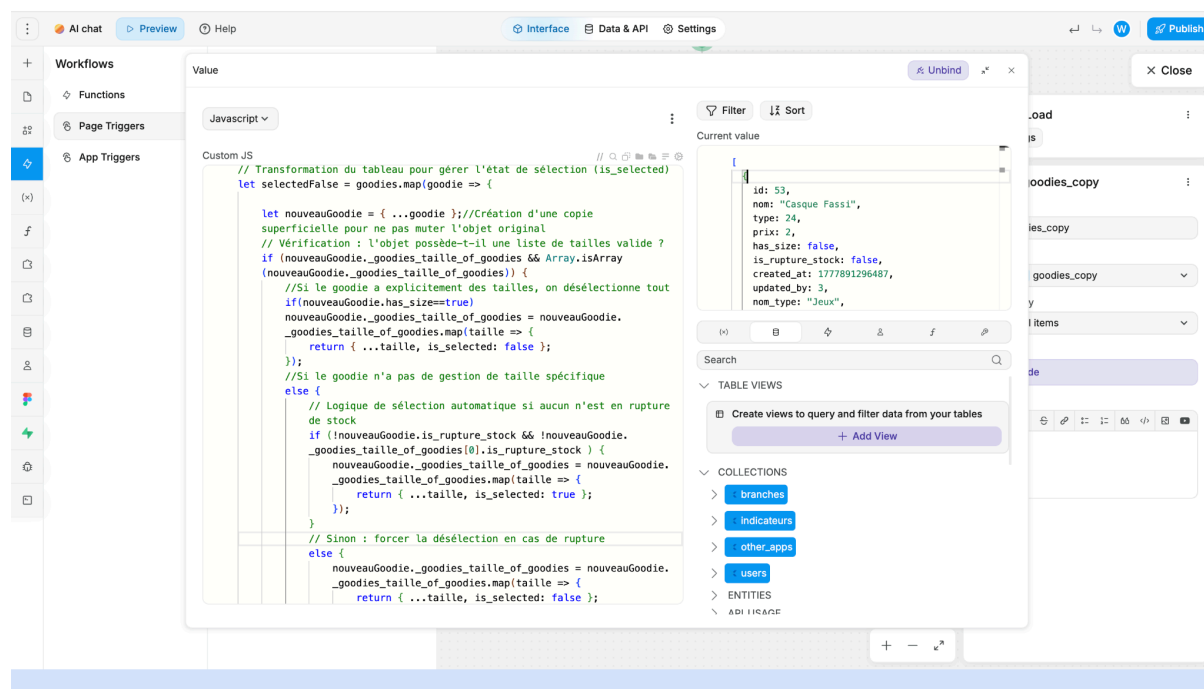
L'écran **Goodies** est plus riche : il présente le catalogue des goodies (objets promotionnels), avec une recherche par nom et un filtre par type (textile, accessoire, etc.). Comme pour les autres, on peut ajouter au panier, valider via un formulaire et recevoir un e-mail. Pour cet écran, j'ai créé moi-même les trois tables nécessaires : `goodies` (l'objet), `goodies_panier_ligne` (les lignes de panier du client) et une table de **tailles**, qui stocke toutes les tailles d'un goodie et permet de gérer la rupture de stock, les tailles étant elles-mêmes référencées dans les *lookup values* (le répertoire centralisé des listes déroulantes de l'application).



C'est sur cet écran que j'ai rencontré ma plus grosse difficulté technique du stage, et je tiens à la détailler car la solution est instructive. Le besoin paraissait simple : pour chaque produit, l'utilisateur devait pouvoir choisir une taille avant de l'ajouter au panier. J'ai d'abord réutilisé le composant **filter chips** (des puces sélectionnables) de **M.Lacroix**. Visuellement, cela fonctionnait : je pouvais bien sélectionner une taille. Mais le composant était conçu en mode « radio » à l'échelle de toute la page : il ne gérait qu'une seule sélection globale. Concrètement, dès que je choisisais une taille sur un produit, la sélection sautait sur tous les autres produits ils étaient tous **dépendants les uns des autres**, ce qui rendait l'écran inutilisable dès qu'il y avait plusieurs goodies.

Après en avoir discuté avec M.Lacroix, nous avons retenu son idée : construire une grande variable qui **recopie l'ensemble de la collection des goodies**, et ajouter à chaque taille un champ booléen **is_selected**. J'ai ensuite écrit un petit algorithme qui, au clic sur une puce, ne met **is_selected** à **true** que pour la taille choisie d'un produit donné, en repassant les autres tailles du même produit à **false**. De cette façon, chaque produit gère sa propre sélection et les goodies redeviennent **indépendants**. Deux points de vigilance : il faut rafraîchir cette variable de copie à chaque chargement de la page (sinon elle reste figée), et bien réinitialiser tous les **is_selected** à **false** au départ, sous peine d'avoir des tailles sélectionnées par défaut

sans raison.



État d'avancement : les deux écrans sont terminés. Les tailles des goodies sont fonctionnelles, la rupture de stock est gérée et affichée, et les e-mails partent correctement.

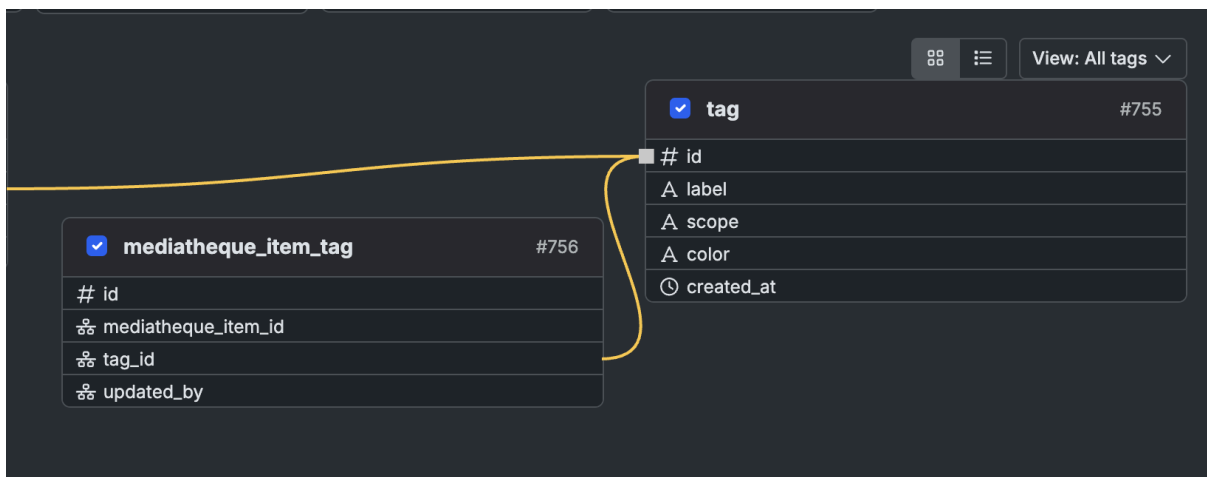
II.2.c. La Médiathèque : gérer fichiers et mots-clés

La Médiathèque appartient elle aussi à l'espace Marketing, mais elle n'a pas de notion de panier : son but est de permettre aux concessionnaires de **retrouver et de télécharger** des logos et des ressources graphiques. Cet écran m'a obligé à manipuler une logique de données un peu plus subtile que les précédents.

L'écran est divisé en deux onglets, l'un pour les **photos**, l'autre pour les **logos**, ce que j'ai réalisé grâce au composant maison *ViewSet NavBar* (une barre de navigation qui bascule entre plusieurs vues à l'aide d'une variable *Active ViewSet*). La distinction entre les deux types n'est pas qu'esthétique : un **logo** peut être constitué de plusieurs fichiers, chacun portant un label décrivant par exemple son format, alors qu'une **photo** ne contient qu'un seul fichier, mais est associée à plusieurs **mots-clés** (des tags) qui facilitent sa recherche.

Côté base de données, cela se traduit par plusieurs tables : *mediatheque_item* pour le média lui-même, *mediatheque_file* pour la liste de ses fichiers, et *mediatheque_item_tag* pour la liste de ses mots-clés. Ce dernier point est important : les mots-clés ne sont pas stockés en texte libre, mais reliés à une table *tag centrale* à toute l'application, dotée d'un champ *scope* qui précise à quelle table appartient le tag (ici, la médiathèque). Cela évite de réinventer les tags pour chaque écran et de les dupliquer.

Base de données Médiathèque :



La difficulté est apparue au moment de la **mise à jour** d'un média (l'API PATCH) : quand l'utilisateur modifie les mots-clés d'une photo, comment savoir lesquels il a ajoutés, lesquels il a supprimés, lesquels sont restés ? Tenter de comparer l'ancienne et la nouvelle liste pour ne toucher que les différences se révélait compliqué et fragile. La solution que j'ai retenue, avec l'aide de l'équipe, est plus radicale mais beaucoup plus fiable : **tout supprimer puis tout ré-insérer**, même si certains mots-clés sont identiques. Pour que cela fonctionne proprement, j'ai eu besoin d'une fonction qui crée dans la table `tag` les mots-clés qui n'existeraient pas encore. C'est la fonction `create_missing_tags`, que je détaille dans la troisième partie car elle constitue un point technique à part entière.

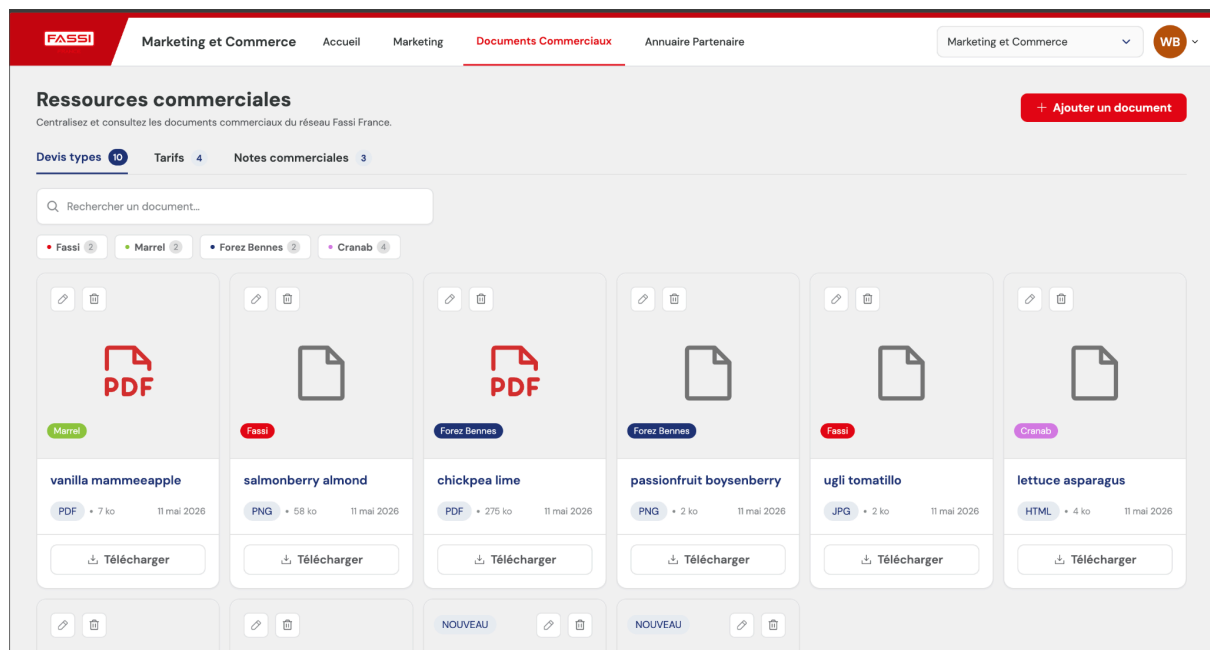
État d'avancement : terminé, avec les deux onglets photos/logos et la gestion complète des mots-clés et des fichiers en back-office.

II.2.d. Les Documents commerciaux

Ce dernier écran de l'espace Marketing permet de consulter et de télécharger les documents commerciaux du groupe. On y retrouve une navigation par onglets, une recherche par nom et un filtre par marque, l'ajout, la modification et la suppression restant réservés au back-office. Les types et les marques sont reliés à la table `lookup_value`.

Cet écran a surtout été l'occasion d'**uniformiser** mes réalisations à la suite des retours de la cliente : reprendre le même style d'onglets que la Médiathèque, faire en sorte que le nombre de documents par marque s'adapte au filtre de type sélectionné, remplacer les icônes « œil » et « télécharger » par un simple lien sur le nom du document, et ajouter des *toasts* de confirmation à l'ajout et à la modification. On m'a aussi demandé de créer un composant *Fuzzy Search* avec une icône loupe intégrée, pour homogénéiser la recherche sur l'ensemble des écrans.

Écran documents commerciaux :



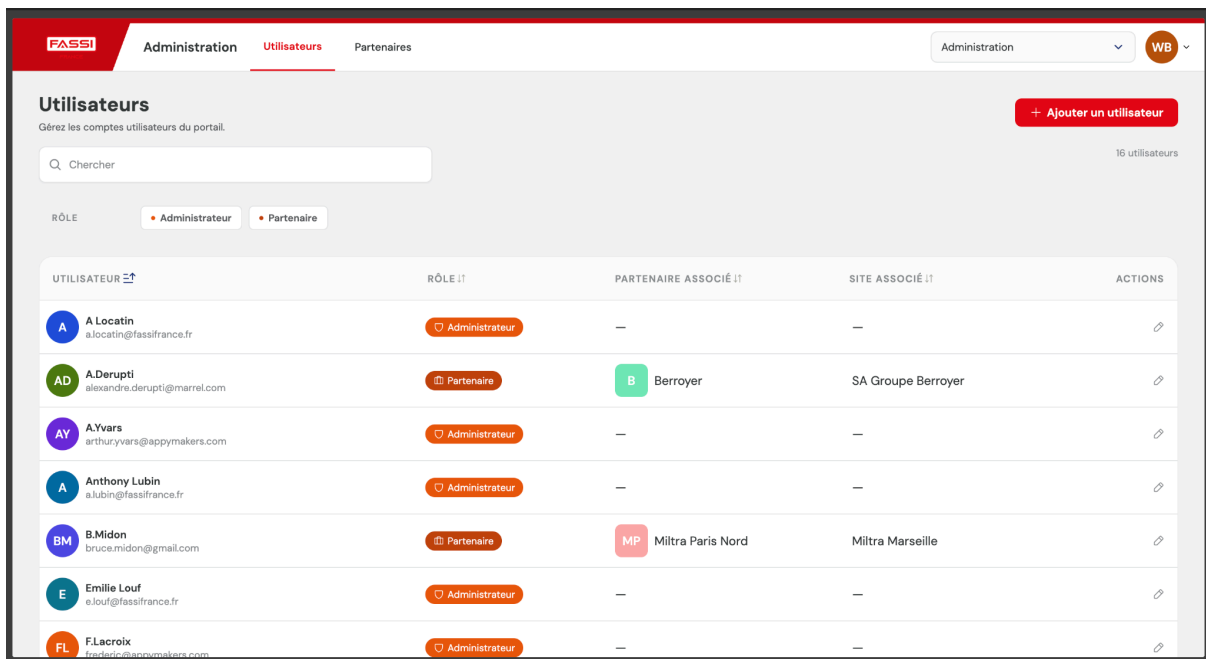
État d'avancement : terminé et harmonisé avec le reste de l'application.

II.3. L'espace Administration

Une fois l'essentiel du Marketing en place, je suis passé aux écrans d'administration, visibles uniquement par les administrateurs. Ces écrans sont moins « grand public » mais essentiels au bon fonctionnement du portail.

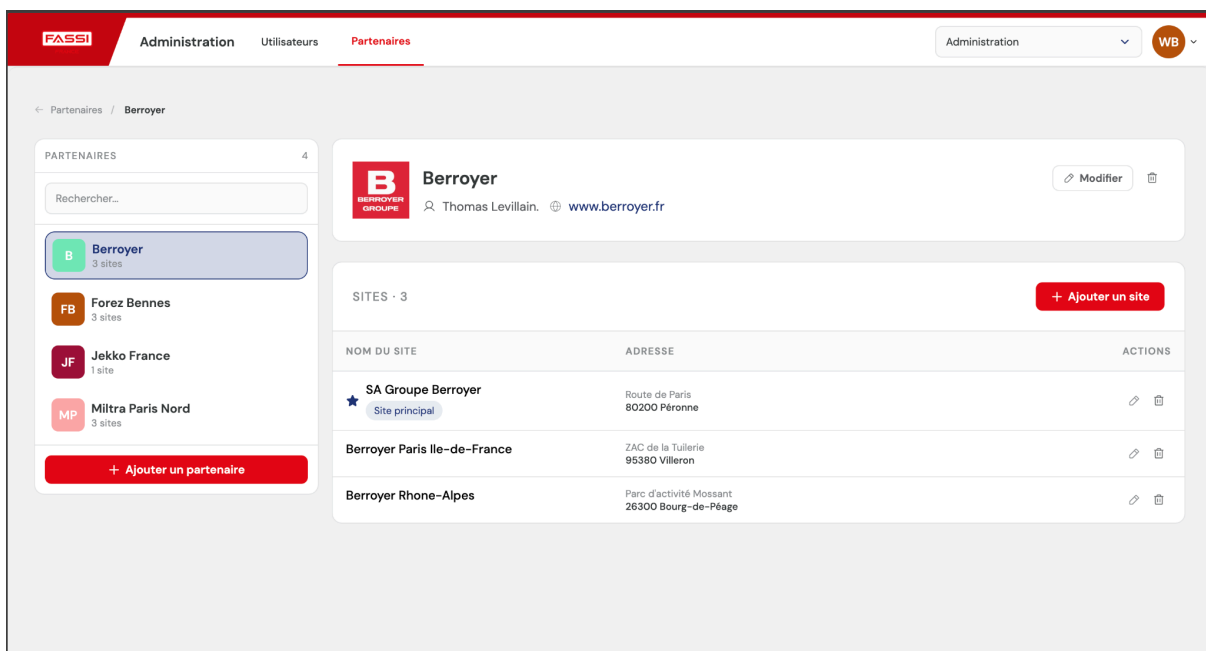
L'écran **Utilisateurs** affiche la liste des comptes et permet de modifier un utilisateur, de changer son rôle (administrateur, partenaire, etc.) ou d'ajouter un nouveau compte — sans définir de mot de passe à la création, celui-ci faisant l'objet d'une procédure dédiée. La table **user** est reliée à la table **rôle**, qui est la pierre angulaire de la gestion des droits décrite plus loin.

Écran utilisateurs :



L'écran **Partenaires** permet de gérer les concessionnaires : les ajouter, les modifier, les supprimer, et gérer leurs différents sites avec la contrainte qu'un partenaire ne peut avoir **qu'un seul site principal**. À la demande de la cliente, j'ai déplacé la modification et la suppression dans un écran de **détail** plutôt que de les laisser directement dans la liste, ce qui rend l'interface plus claire.

Écran Partenaires :



Enfin, l'écran **Annuaire des partenaires** affiche le réseau sous forme de répertoire consultable, avec une recherche et un filtre par **expertise**. Pour gérer ces expertises, j'ai réutilisé exactement la même stratégie que pour les mots-clés de la médiathèque : tout supprimer puis tout ré-insérer. C'est un bon exemple de réutilisation d'une solution éprouvée, et c'est aussi pour cela que je n'ai pas rencontré de grande difficulté sur ces trois écrans.

État d'avancement : les trois écrans d'administration sont terminés.

II.4. L'espace Service Après-Vente

En fin de stage, j'ai abordé l'espace SAV, ce qui m'a permis de toucher à des écrans un peu différents des catalogues que j'avais l'habitude de produire.

L'écran **Documents techniques** suit une logique proche des documents commerciaux : un document est relié à une catégorie, à une marque et à plusieurs tags (avec le scope `doc_technique_tag` dans la table `tag` centrale). On peut le rechercher par onglet de marque ou de catégorie, ou en saisissant son nom, le visualiser, et le gérer (ajout, modification, suppression) en back-office.

État d'avancement : écrans maquetés et développés ; ils font partie des éléments en cours de finalisation avec les derniers retours du client.

II.5. Une brique transversale : la gestion des droits

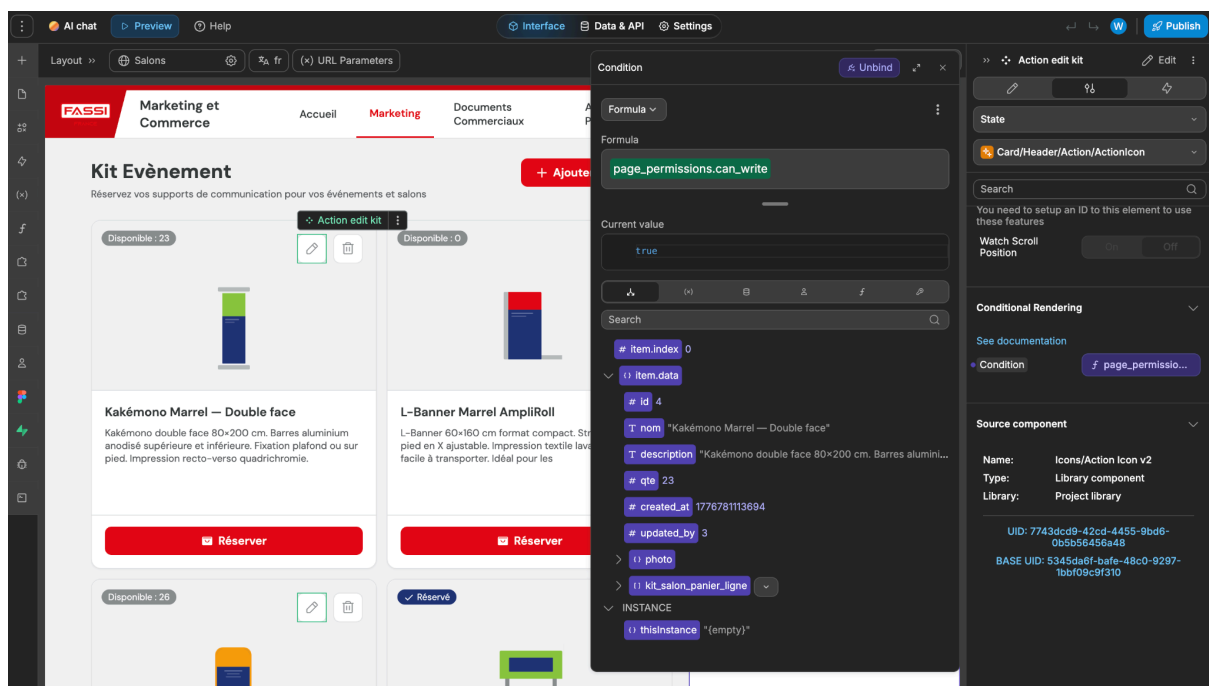
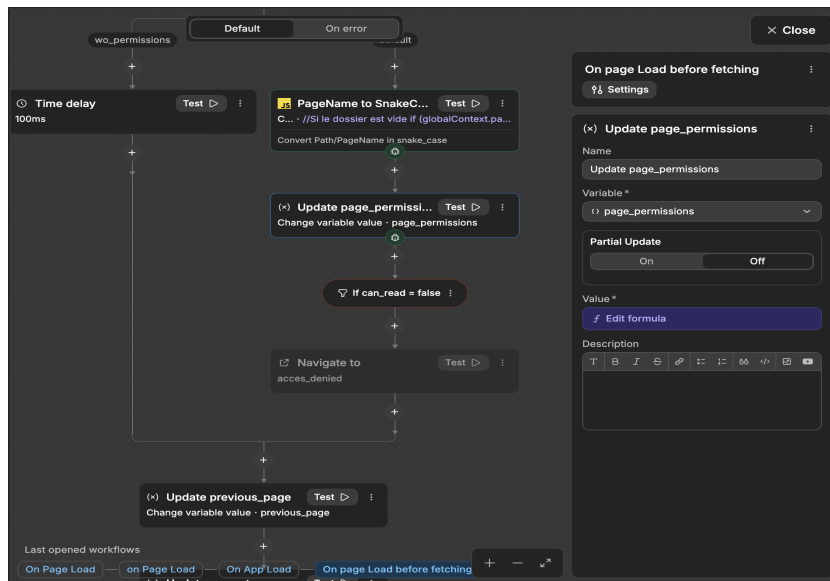
Un dernier point mérite d'être présenté à part, car il ne concerne pas un écran mais traverse toute l'application : la **gestion des droits**. Sur ConcessLink, tous les utilisateurs ne voient pas la même chose. Un administrateur a accès au back-office, un partenaire non.

Cette gestion repose sur une table `rôle` dans Xano. Au-delà du nom du rôle et de quelques réglages, l'essentiel se joue dans des colonnes préfixées par `p_` ou `e_`. Les colonnes en `p_` servent au **front-end** : elles indiquent, page par page, si l'utilisateur peut écrire, lire seulement, ou n'a aucun droit. Leur nom suit strictement la forme `p_dossier_nom_page` la moindre faute de frappe casse toute la mécanique, ce qui m'a appris à être rigoureux sur le nommage.

Le fonctionnement côté WeWeb est le suivant : un workflow transforme d'abord le nom de la page en `snake_case` pour qu'il corresponde exactement au nom de la colonne de la table `rôle`. Il vérifie ensuite si la colonne correspondante existe : si oui, il renvoie un objet JSON du type `{ page: "nom_page", can_read: true, can_write: false }` ; si la page n'est pas trouvée, il renvoie

une erreur et l'utilisateur n'y a tout simplement pas accès. Ce JSON est stocké dans une variable qui pilote ensuite l'affichage : sur les boutons réservés aux administrateurs, j'ai posé des conditions d'affichage pour qu'un partenaire ne les voie même pas.

La même logique existe côté **API**, avec les colonnes préfixées en **e_** (sous la forme **e_nom_table**) : chaque API vérifie si l'utilisateur a le droit de lire ou d'écrire et, dans le cas contraire, renvoie une erreur qui bloque la requête. C'est une double sécurité importante : cacher un bouton dans l'interface ne suffit pas, il faut aussi protéger l'API elle-même, sans quoi un utilisateur malintentionné pourrait l'appeler directement. Vous trouverez ci-dessous des illustrations :



II.7. État d'avancement global du projet

À ce jour, le projet ConcessLink est **abouti** : l'ensemble des écrans dont j'avais la charge est développé, fonctionnel et relié à la gestion des droits. Nous sommes désormais en attente des **derniers retours du client** pour peaufiner quelques informations et finaliser certains détails. Les phases de tests, menées en interne sur le principe de « se mettre dans la peau d'un débutant qui essaie de faire bugger l'application », ont permis de fiabiliser ces écrans avant cette ultime relecture.

III. La réalisation de composants réutilisables et de logique back-end

Si la deuxième partie présentait des écrans complets, cette troisième partie se concentre sur un travail plus « transversal » et plus technique : la création de **composants réutilisables** et d'une **fonction back-end** que l'on retrouve à plusieurs endroits de l'application. C'est, à mes yeux, la facette la plus intéressante du stage, car elle dépasse l'assemblage visuel pour toucher à du vrai développement.

Un mot d'abord sur l'intérêt des composants. Un composant est un bloc d'interface préconfiguré et indépendant, qui réunit une structure visuelle et une logique interne (variables privées, workflows, déclencheurs). Il repose sur le principe d'un « modèle maître » que l'on peut dupliquer à l'infini : toute modification du modèle se répercute instantanément sur toutes ses copies. L'intérêt est double : garantir la **cohérence** de l'interface et accélérer la **maintenance**. Sur ConcessLink, deux familles de composants coexistent : ceux que l'on construit directement dans WeWeb (no-code), et ceux que l'on **code en Vue.js** puis que l'on importe — ces derniers servant à couvrir les besoins que les éléments natifs de WeWeb ne savent pas faire.

III.1. Le composant Active Filter Chips (no-code)

Le premier composant que j'ai réalisé est l'**Active Filter Chips**. Son rôle est d'afficher, sous forme de petites puces (« chips »), la liste des filtres actuellement actifs sur une page, et de permettre à l'utilisateur de retirer un filtre d'un simple clic sur sa croix.

Le fonctionnement repose sur une variable locale `active_filters` de type tableau : c'est une liste d'objets, chaque objet contenant un identifiant, un libellé et une valeur. Le composant affiche une puce par objet présent dans cette liste ; les chips dépendent donc entièrement de cette variable. Pour le piloter depuis la page, j'ai exposé plusieurs workflows : `add_filter` ajoute un filtre, `remove_filter` en retire un, et `set_active_filters` initialise ou remplace

l'ensemble. À l'inverse, quand l'utilisateur clique sur la croix d'une puce, le composant émet un événement (`delete_value`) renvoyant le libellé et la liste des valeurs restantes, ce qui permet à la page parente de **mettre à jour ses données** en conséquence.

J'ai fait évoluer ce composant à la suite des retours : dans sa version finale (rebaptisée *Dynamic Filter Chips*), il s'active automatiquement dès que l'utilisateur sélectionne une option dans l'un des filtres de la page, gère silencieusement la mise à jour des données, et propose un réglage de taille (M ou S). J'y ai même ajouté la possibilité de saisir et de valider un libellé personnalisé, ainsi qu'un workflow de réinitialisation.

The image shows a user interface for filtering data. It consists of two main sections: 'Type' and 'Marque'. The 'Type' section has a dropdown menu currently showing 'VTT'. The 'Marque' section has a multi-select menu with three items: 'Canyon', 'Look', and 'Trek', each with a close button (X). Below these sections is a horizontal bar that summarizes the selected filters: 'Type : VTT' and 'Marque : Canyon Look Trek', with each item having a close button (X).

C'est avec ce composant que j'ai compris la **communication entre un composant et sa page** : un composant ne doit pas modifier directement les données de la page, il se contente d'émettre des événements, et c'est la page qui réagit. Cette séparation des responsabilités est exactement la même logique que celle vue en cours sur les architectures front-end.

III.2. Le composant Input Number (codé en Vue.js)

Le composant **Input Number** est la réalisation technique dont je suis le plus fier, car c'est un composant que j'ai entièrement **codé en Vue.js** avant de l'intégrer à WeWeb. Le besoin était précis : WeWeb propose bien un champ de saisie numérique natif, mais il ne savait pas mettre en forme un nombre « à la française » en temps réel — c'est-à-dire afficher des espaces comme séparateurs de milliers (par exemple **1 250 000**), gérer un symbole monétaire, choisir le nombre de décimales — tout en gardant un comportement correct du curseur pendant la frappe. C'est typiquement le genre de cas où l'on quitte le no-code pour écrire du code.

Le cheminement technique

Comme WeWeb utilise Vue 3 sous le capot, un composant codé est en réalité un composant Vue tout à fait standard, simplement chargé dans un environnement particulier. Le cycle de travail, que j'ai appris pendant le stage et que j'ai même documenté pour l'entreprise, est le suivant : on initialise le projet avec l'outil en ligne de commande de WeWeb (`npm init @weweb/component`), on développe et on **teste en local** (`npm run serve`, en chargeant le composant dans le « Dev Editor » de WeWeb), puis on **pousse le code sur GitHub**, et enfin on **relie le dépôt GitHub** au tableau de bord WeWeb pour pouvoir glisser-déposer le

composant dans l'application publiée. Chaque nouveau push déclenche automatiquement une reconstruction côté WeWeb.

Un composant codé pour WeWeb s'articule autour de deux fichiers. Le fichier `ww-config.js` déclare toutes les propriétés que l'on pourra régler **sans coder** depuis l'éditeur : pour mon Input Number, j'ai exposé la valeur initiale, le placeholder, le caractère obligatoire ou non, l'affichage et la position d'un symbole monétaire, le nombre de décimales, le choix des séparateurs de milliers et de décimales, ainsi qu'une option de *debounce*. Le fichier `wwElement.vue` contient, lui, la logique réelle du composant.

Le cœur de la logique

Deux mécanismes m'ont demandé le plus de réflexion.

Le premier est l'**exposition de la valeur** au reste de l'application. Grâce à `wwLib.wwVariable.useComponentVariable`, le composant publie une variable `value` que n'importe quel autre élément de la page peut lire (par exemple pour l'envoyer dans un formulaire). Il a fallu veiller à garder cette variable exposée toujours synchronisée avec ce que l'utilisateur voit à l'écran, y compris quand la valeur initiale change de l'extérieur — d'où un `watch` sur `content.value`.

Le second, et de loin le plus délicat, est la **gestion du curseur**. Lorsque l'utilisateur tape un nombre, ma méthode `handleInput` nettoie d'abord la saisie pour ne garder que les chiffres et le séparateur décimal, limite le nombre de décimales, puis ré-applique les espaces de milliers. Le problème, c'est qu'en insérant ces espaces, je modifie la chaîne affichée, et le curseur « saute » à un endroit imprévu — une expérience très désagréable à l'usage. Pour le corriger, je compte le nombre de caractères « utiles » (hors espaces) situés avant le curseur, puis je reparcours la chaîne reformatée pour replacer le curseur exactement au bon endroit. Une fois la mise en forme faite, je convertis la valeur affichée en une **valeur technique** (en remplaçant le séparateur décimal par un point pour obtenir un nombre exploitable), je mets à jour la variable exposée et j'émet l'événement `change`.

J'ai aussi ajouté une option de *debounce* qui, si elle est activée, attend 500 millisecondes après la dernière frappe avant de déclencher la mise à jour, ce qui évite de surcharger l'application quand on tape vite.

Ce composant a connu plusieurs versions (j'ai commencé par une version simple de formatage des milliers, avant d'ajouter la devise, les séparateurs configurables et le *debounce*).

C'est une bonne illustration de la **montée en complexité progressive** d'un développement, et cela m'a aussi fait pratiquer le versionnage :

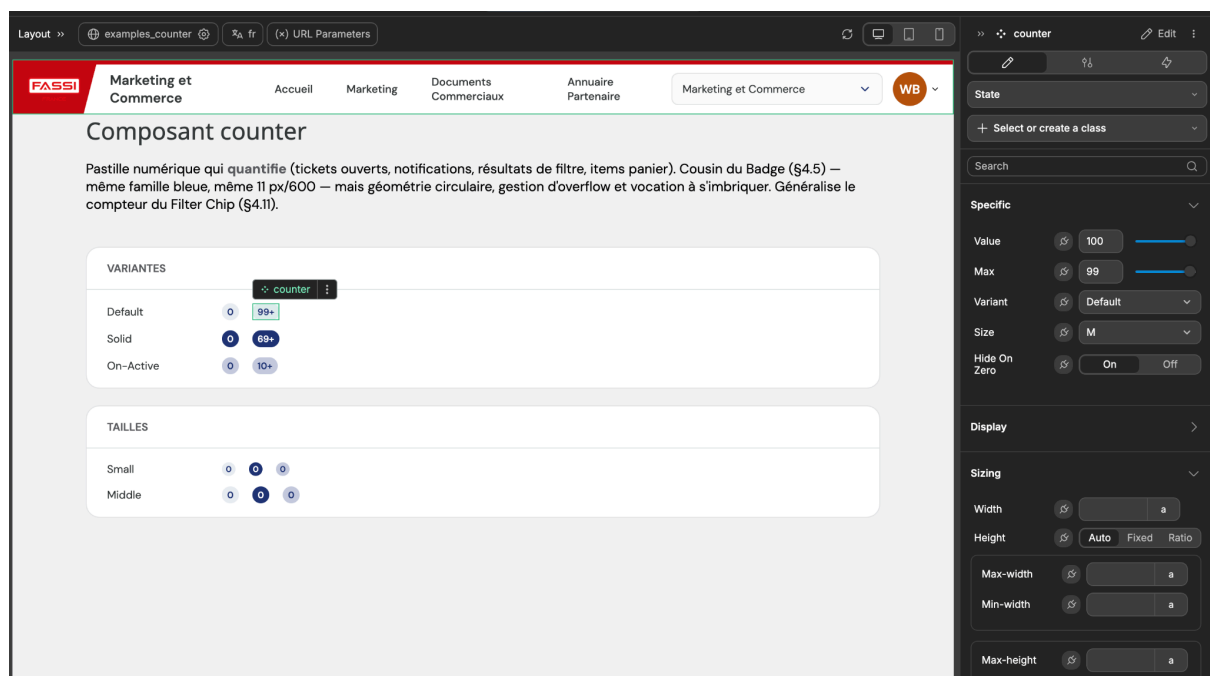
sur WeWeb, tant qu'on ne change pas la « version active » dans le tableau de bord, l'application en production n'est pas modifiée, même si le code est déjà sur GitHub — une sécurité que j'ai trouvée plutôt élégante.

III.3. Le composant Counter

Le composant **Counter** est plus modeste mais utile : c'est une pastille numérique qui sert à quantifier des éléments (par exemple le nombre d'articles dans un panier ou de documents dans un onglet). Il est conçu pour s'imbriquer dans d'autres éléments et s'élargit automatiquement quand le nombre comporte plusieurs chiffres. Je l'ai réalisé à partir d'une spécification HTML fournie par Frédéric, en renommant proprement les paramètres et en le documentant sur GitBook.

Ses réglages illustrent bien le soin apporté aux petits composants : une valeur affichée (**Value**), un maximum (**Max**, fixé à 99 par défaut) au-delà duquel la pastille affiche un « + » plutôt qu'un grand nombre disgracieux, une taille (S ou M), un style visuel (**Variant** : **default** pour les surfaces claires, **on-active** pour les fonds bleus actifs, **solid** pour les notifications) et une option **Hide On Zero** qui masque automatiquement la pastille lorsque la valeur vaut zéro. Ce dernier composant a remplacé un ancien compteur de la médiathèque, que l'on a ensuite pu supprimer.

Illustration du composant Counter :



III.4. La fonction back-end `create_missing_tags` (Xano)

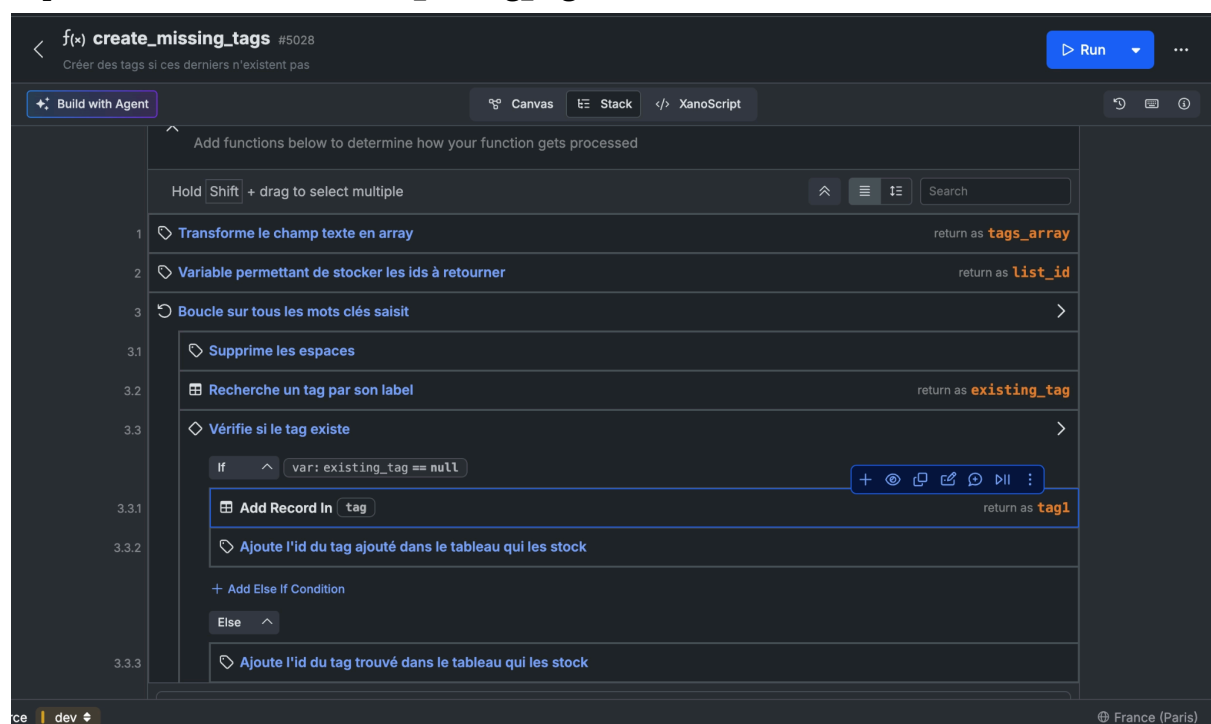
Pour terminer, je veux revenir sur le point technique back-end le plus marquant, déjà évoqué dans la partie sur la Médiathèque : la fonction `create_missing_tags`.

Le contexte est le suivant. Les mots-clés des médias (et, plus tard, les expertises des partenaires) ne sont pas stockés en texte libre : ils sont reliés à une table `tag` **centrale** à toute l'application, partagée par plusieurs écrans grâce à un champ `scope` qui indique à quelle table le tag se rapporte. Cette centralisation évite les doublons et garantit que, si deux médias utilisent le mot-clé « grue », ils pointent bien vers le **même** enregistrement.

Le problème se pose au moment d'enregistrer les tags d'un média. Comme je l'ai expliqué, j'ai choisi la stratégie « tout supprimer puis tout ré-insérer » pour les liens entre le média et ses tags, car elle est bien plus fiable que de calculer les différences. Mais cette approche suppose que **chaque mot-clé existe déjà** dans la table centrale. Or, rien n'empêche un utilisateur de saisir un mot-clé tout neuf qui n'a jamais été utilisé ailleurs.

C'est précisément le rôle de `create_missing_tags`. Cette fonction reçoit la liste des mots-clés saisis ainsi que leur scope. Pour chaque mot-clé, elle vérifie s'il existe déjà dans la table `tag` pour ce scope ; si oui, elle récupère son identifiant ; sinon, elle le crée puis récupère l'identifiant du nouveau tag. Elle renvoie finalement la **liste complète des identifiants** de tags, prête à être reliée au média. Le reste de l'API de mise à jour devient alors très simple : on supprime les anciens liens, puis on recrée les liens vers les identifiants renvoyés par la fonction.

Capture de la fonction `create_mising_tags` :



Cette fonction est un bon exemple de **factorisation** : plutôt que de répéter la même logique dans l'API de la médiathèque, puis dans celle des partenaires, je l'ai isolée une fois pour toutes dans une fonction réutilisable. C'est exactement le réflexe que l'on nous enseigne en cours « ne pas se répéter », mais appliqué cette fois à un environnement back-end visuel comme Xano. Je trouve d'ailleurs intéressant de constater que, même sans écrire de SQL à la main, les mêmes bonnes pratiques de conception restent valables.

IV. Bilan des compétences mises en œuvre et acquises

Au-delà du travail technique présenté dans les parties précédentes, ce stage doit aussi se lire comme une étape de ma formation. Cette dernière partie propose donc un bilan plus analytique : je relis mon expérience chez AppyMakers à la lumière du référentiel 2022 du BUT Informatique, parcours Réalisation d'applications (RA), en m'appuyant sur les apprentissages critiques (AC) des différentes compétences. Comme je suis en deuxième année, le niveau attendu est le niveau 2 de chaque compétence, et c'est donc surtout à ce niveau que je situe mes acquis. Ce bilan fait écho, sous une forme plus détaillée, aux pages de mon portfolio consacrées au stage.

Je précise mon ressenti général, que j'expliciterais compétence par compétence : si le stage a touché plus ou moins fortement aux six compétences du référentiel, c'est sur trois d'entre elles que j'ai senti une vraie progression **Réaliser (UE1)**, **Optimiser (UE2)** et **Gérer (UE4)**. Ce n'est pas un hasard : ce sont exactement les compétences au cœur du métier de développeur que j'ai exercé pendant treize semaines.

IV.1. La gestion de projet et l'organisation des missions

Avant d'entrer dans le détail des compétences techniques, je veux revenir sur l'organisation de mes missions, car c'est elle qui a structuré tout mon stage. Comme je l'ai expliqué en première partie, AppyMakers suit une méthode agile adaptée à une équipe de six personnes : une réunion le matin où M. Lacroix répartissait ou réajustait les tâches, un point informel l'après-midi, et un suivi macroscopique sur Notion avec des colonnes « à faire / en cours / terminé ». Beaucoup de priorités se décidaient à l'oral, ce qui m'a obligé à être autonome dans ma propre organisation : j'ai pris l'habitude de noter systématiquement les consignes sur Notion.

Ce fonctionnement m'a fait découvrir concrètement ce que le référentiel appelle, pour la compétence **Conduire**, « mettre en place les outils de gestion de projet » et « définir et mettre en œuvre une démarche de suivi de projet ».

J'ai surtout compris une chose que les cours ne peuvent pas vraiment transmettre : un projet réel avancé par **itérations** et par retours clients successifs. Les réunions de relecture avec la cliente de Fassi France ont été, à ce titre, les jalons les plus structurants : un écran « qui fonctionne » n'était jamais un écran « validé », et chaque relecture déclenchait une nouvelle palettes d'ajustements à intégrer.

Le tableau ci-dessous synthétise le calendrier de mes missions et les compétences mobilisées à chaque phase. Il sert aussi à montrer la logique de progression de mon stage : on m'a confié des tâches de plus en plus autonomes, de l'écran « modèle » accompagné jusqu'aux composants techniques réalisés seul.

Période	Missions principales	Compétences mobilisées
Semaine 1 (avril)	Intégration, autoformation Xano/WeWeb, application « bac à sable »	Collaborer (C6), Réaliser (C1)
Semaines 2–3	Écran Brochures et Suppots salon : back-end Xano, front WeWeb, panier, e-mail, back-office ; 1 ^{re} relecture cliente	Réaliser (C1), Gérer (C4), Conduire (C5)
Semaine 4	Écran Goodies (difficulté is_selected) et écran Médiathèque (modèle de données item/fichiers/tags), fonction create_missing_tags	Réaliser (C1), Gérer (C4), Optimiser (C2)
Semaines 5–6	Documents commerciaux, harmonisation des écrans	Réaliser (C1), Optimiser (C2)
Semaine 7	Composant Input Number codé en Vue.js, composant Counter (Low-Code)	Réaliser (C1), Optimiser (C2)
Semaine 8	Espace Administration (Utilisateurs, Partenaires, Annuaire), composant Header, brique transversale gestion des droits	Réaliser (C1), Gérer (C4), Administrer (C3)
Semaine 9	Espace SAV (Écran : Documents techniques et Accueil SAV), phase de tests, intégration des derniers retours client, rédaction du rapport	Réaliser (C1), Conduire (C5), Collaborer (C6)

IV.2. Compétence « Réaliser » (UE1) : du code écrit à la main à l'application complète

C'est la compétence sur laquelle j'ai le plus progressé, et c'est aussi celle où j'avais, je crois, le plus de marge. Mes moyennes de première année sur l'UE1 *Réaliser* tournaient autour de 11,5 puis 12 sur 20 la plus faible de mes compétences « cœur de développement ». Le stage a été l'occasion de la consolider de manière tangible, car je suis passé d'exercices encadrés à une **vraie application en production**.

Le référentiel place au niveau 2 de Réaliser le fait de « partir des exigences et aller jusqu'à une application complète ». C'est exactement le schéma que j'ai déroulé sur chaque écran de ConcessLink : élaborer et implémenter les spécifications à partir d'un cahier des charges et des retours de la cliente, de la table PostgreSQL jusqu'à l'interface WeWeb. La répétition de ce cycle complet back-end, API CRUD, front-end, back-office sur une dizaine d'écrans est ce qui m'a fait gagner en aisance et en rapidité.

J'ai aussi appris à « adopter de bonnes pratiques de conception et de programmation » dans un environnement “no-code”, ce qui n'allait pas de soi. Le réflexe le plus marquant a été la factorisation : plutôt que de reproduire la même logique d'écran en écran, j'ai cherché à isoler ce qui pouvait l'être (composants réutilisables, fonction back-end partagée). La gestion des retours clients m'a par ailleurs sensibilisé à l'ergonomie et à l'accessibilité : tailles de cartes, couleurs de boutons et de puces, ouverture des PDF dans un nouvel onglet, remplacement des notifications éphémères par des fenêtres modales... autant de détails qui paraissent secondaires mais qui font la différence pour l'utilisateur final.

Enfin, la phase de tests menée en fin de stage « se mettre dans la peau d'un débutant qui essaie de faire bugger l'application » relève directement de « vérifier et valider la qualité de l'application par les tests ». Ce n'étaient pas des tests automatisés comme on peut en écrire en cours de qualité de développement, mais une démarche de test fonctionnel rigoureuse, et j'ai pris conscience à quel point elle est indispensable avant une livraison client.

IV.3. Compétence « Optimiser » (UE2) : appliquer l'algorithmique en contexte réel

J'avais des moyennes plutôt correctes en *Optimiser* (autour de 13–14 sur 20 en première année), mais cette compétence restait pour moi très théorique : algorithmes de tri, structures de données étudiées « pour elles-mêmes ». Le stage m'a fait sentir une progression d'une autre nature : non pas de meilleures notes, mais la capacité à **mobiliser ces raisonnements pour résoudre un vrai problème**, ce qui me semble être l'enjeu réel de cette compétence.

Deux situations l'illustrent particulièrement. La première est la sélection des tailles sur l'écran Goodies : pour rendre chaque produit indépendant, j'ai dû concevoir une structure de données adaptée une copie de la collection enrichie d'un champ booléen `is_selected` par taille puis écrire un petit algorithme qui ne met à `true` que la taille choisie d'un produit donné en repassant les autres à `false`. Choisir cette structure plutôt qu'une comparaison fragile de listes, c'est précisément « choisir des structures de données adaptées au problème ». La seconde est la gestion du curseur dans mon composant Input Number : pour éviter que le curseur ne « saute » quand on insère les séparateurs de milliers, j'ai dû compter les caractères utiles avant le curseur, puis le repositionner dans la chaîne reformatée.

C'est l'application la plus aboutie d'une « technique algorithmique adaptée à un problème complexe » que j'aie eu à écrire de tout le stage.

J'ai aussi touché à l'optimisation des performances avec l'option de **debounce** du même composant (attendre 500 ms après la dernière frappe pour éviter de surcharger l'application), qui rejoint la préoccupation d'anticiper le comportement d'un code à l'usage. Je reste lucide : le stage n'était pas un stage d'optimisation pure, et je n'ai pas mesuré finement des métriques de performance. Mais le lien entre l'algorithmique de cours et le développement concret, lui, est devenu beaucoup plus clair.

IV.4. Compétence « Gérer » (UE4) : la modélisation et l'exploitation des données

Gérer était déjà l'une de mes compétences les plus solides en formation (de 12,7 en S1 à 14,8 en S2), et le stage a confirmé puis approfondi cette aisance. Une grande partie de mon travail sur Xano consistait en effet à concevoir et exploiter des bases de données relationnelles PostgreSQL, ce qui place cette compétence au centre de mon stage.

Au niveau 1, j'ai constamment « conçu une base de données relationnelle à partir d'un cahier des charges client », par exemple en créant moi-même les trois tables de l'écran Goodies (`goodies`, `goodies_panneau_ligne` et la table de tailles) et « interrogé et mis à jour ces bases » via les API CRUD, y compris en corrigeant des jointures (le *left join* des Supports salon qui ne renvoyait pas les bonnes données).

Mais c'est surtout au niveau 2 que j'ai senti une montée en compétence. La table `tag` centrale, partagée par plusieurs écrans grâce à son champ `scope`, est un vrai choix d'**optimisation du modèle de données de l'entreprise** : en mutualisant les mots-clés au lieu de les dupliquer écran par écran, on évite les doublons et on garantit la cohérence des données. La fonction `create_missing_tags`, mon point technique back-end, prolonge cette logique : factoriser une fois pour toutes la création des tags manquants pour la réutiliser dans la médiathèque puis dans les partenaires. J'ai par ailleurs manipulé des données de natures variées médias, fichiers multiples, tags, **lookup values** ce qui rejoint « manipuler

des données hétérogènes ». Tout cela sans écrire une ligne de SQL à la main, ce qui m'a fait réaliser que les bonnes pratiques de conception de bases de données restent strictement les mêmes, qu'on travaille en SQL ou dans un outil visuel comme Xano.

IV.5. Les autres compétences : Administrer, Conduire, Collaborer

Trois compétences ont été mobilisées plus ponctuellement, mais méritent d'être mentionnées.

Administrer (UE3) est celle que j'ai le moins exercée c'est cohérent avec un poste de développeur d'applications, et non d'administrateur système. Je l'ai toutefois touchée à travers la brique transversale de **gestion des droits** : la double sécurité que j'ai mise en place (cacher les boutons côté WeWeb et vérifier les droits côté API avec les colonnes **e_**) relève de « sécuriser les services et les données d'un système ». J'ai compris à cette occasion qu'une protection purement visuelle ne suffit jamais, puisqu'un utilisateur malveillant peut appeler directement l'API. Le développement d'API connectant front et back-end touche également à la conception « d'applications communicantes ».

Conduire (UE5), en partie traitée plus haut, m'a fait surtout progresser sur la « formalisation des besoins du client et de l'utilisateur » à travers les cahiers des charges Notion et les réunions de relecture, ainsi que sur la compréhension des phases d'un cycle de développement.

Collaborer (UE6) est sans doute, avec **Réaliser**, la compétence dont ce stage a le plus enrichi le versant « savoir-être ». Découvrir le fonctionnement interne d'une ESN organisée en deux pôles correspond à « comprendre la structure et la dimension de l'informatique dans une organisation » m'intégrer à une équipe de six personnes en télétravail le jeudi, communiquer via Teams et rendre compte de mon avancement chaque jour relève de « rendre compte de son activité professionnelle ». Ce rapport et la soutenance qui l'accompagne en sont, d'ailleurs, le prolongement.

IV.6. Synthèse auto-évaluative

Avec le recul, je pense que la principale réussite de ce stage est d'avoir transformé des compétences encore « scolaires » en compétences appliquées. **Réaliser**, qui était ma compétence la plus fragile, est devenue la plus naturelle à force de dérouler le cycle complet de développement sur de vrais écrans ; c'est sur elle que la progression a été la plus franche. **Optimiser** a quitté le terrain purement théorique pour devenir un outil de résolution de problèmes concrets, et **Gérer** a confirmé un point fort en l'enrichissant de vraies décisions de modélisation.

Je reste honnête sur mes limites : je n'ai pas vraiment touché à l'administration système ni à l'optimisation fine des performances, et certaines de mes solutions (la stratégie « tout supprimer puis tout réinsérer » pour les tags) ont été trouvées avec l'aide précieuse de l'équipe plutôt que seul. Mais c'est aussi cela, je crois, l'apprentissage d'un premier stage : savoir où l'on est compétent, où l'on a progressé, et où il reste du chemin. Ce bilan rejoint l'analyse, plus visuelle et plus synthétique, que je propose dans la partie « stage » de mon portfolio.

Conclusion générale

À ce stade de mon stage, après une dizaine de semaines passées chez AppyMakers, je peux déjà dresser un bilan que je qualifierais sans hésiter de très positif. J'ai contribué, du back-end Xano jusqu'au front-end WeWeb, à la reconstruction du portail ConcessLink pour le réseau Fassi France : une dizaine d'écrans métiers complets sur l'espace Marketing & Commerce, des écrans d'administration et de SAV, plusieurs composants réutilisables dont un composant Input Number entièrement codé en Vue.js et une fonction back-end factorisée, [create_missing_tags](#).

La quasi-totalité des écrans dont j'ai la charge est aujourd'hui développée, fonctionnelle et reliée à la gestion des droits ; il me reste à finaliser l'espace SAV et à intégrer les derniers retours du client d'ici la fin du stage. Au-delà de ces livrables, l'acquis le plus durable est ailleurs : j'ai appris à dérouler un cycle de développement complet sur un vrai projet client, à composer avec des retours successifs, et à travailler avec la rigueur qu'exige une livraison en production.

Les attentes que je formulais en introduction ont été pleinement satisfaites. Je souhaitais participer activement à la création d'applications concrètes : non seulement j'y ai participé, mais j'ai pris en charge des écrans de bout en bout. Je voulais découvrir le fonctionnement d'une équipe de développement agile : l'agilité pragmatique d'AppyMakers, ses stand-ups quotidiens et ses itérations rythmées par les relectures clientes m'ont fait toucher du doigt la réalité du travail en équipe. Je voulais enfin apprendre de nouvelles technologies, et la découverte du no-code / low-code a largement dépassé ce que j'imaginais. La question que je me posais au départ : comment nos cours de programmation s'appliquent-ils dans un environnement visuel ? A trouvé sa réponse : la logique sous-jacente est strictement identique. Modélisation de bases de données relationnelles, raisonnement algorithmique, appels d'API sécurisés, factorisation du code, séparation des responsabilités... toutes ces notions vues à l'IUT se sont révélées tout aussi essentielles dans Xano et WeWeb que dans un développement « classique ».

C'est d'ailleurs l'occasion d'une appréciation constructive de la formation reçue. Le BUT Informatique parcours RA m'a fourni des fondamentaux solides bases de données,

algorithmique, développement web, conception qui ont constitué un socle directement réinvestissable.

Les compétences sur lesquelles j'ai le plus progressé en stage (Réaliser, Optimiser, Gérer) sont précisément celles que la formation avait commencé à construire. Le stage a surtout révélé que les cours ne peuvent transmettre que partiellement la dimension humaine et itérative d'un projet, l'importance des tests avant livraison, et l'art de transformer un besoin client parfois flou en spécifications exploitables. Avec le recul, je perçois aussi mieux la valeur des cours que je trouvais abstraits, comme la gestion de projet ou la communication professionnelle.

Sur le plan de mon projet professionnel, ce stage a joué un rôle de confirmation. Il conforte mon choix du parcours Réalisation d'applications : c'est bien le développement, et la conception qui l'entoure, qui me passionnent. Il m'a aussi montré qu'une petite structure à taille humaine, où l'on a une vision globale des projets et un contact direct avec le client, correspond à un cadre dans lequel je m'épanouis. Cette expérience me conforte enfin dans mon souhait de poursuivre en troisième année de BUT en alternance, afin de prolonger cette immersion en entreprise tout en continuant à monter en compétence. Découvrir l'écosystème low-code, encore peu abordé en cours mais en pleine expansion sur le marché, est un atout que je compte bien valoriser dans la suite de mon parcours. Pour un premier contact prolongé avec le monde professionnel, ce stage se révèle déjà, à quelques jours de son terme, une expérience aussi formatrice que motivante.

Sources et Sitographie

1. Ressources de l'entreprise d'accueil

AppyMakers (2026). *Site de AppyMakers*. [En ligne]. Disponible sur : <https://www.appymakers.com/> (Consulté en mai et juin 2026).

AppyMakers (2026). *Documentation technique et guides internes*. [En ligne]. Disponible sur : <https://doc.appymakers.com/> (Consulté tout au long du stage).

2. Documentation Technique et Technologique

WeWeb (2026). *WeWeb Official Documentation*. [En ligne]. Disponible sur : <https://docs.weweb.io/> (Consulté en mai et juin 2026).

Xano (2026). *Xano Official Documentation*. [En ligne]. Disponible sur : <https://docs.xano.com/> (Consulté tout au long du stage).

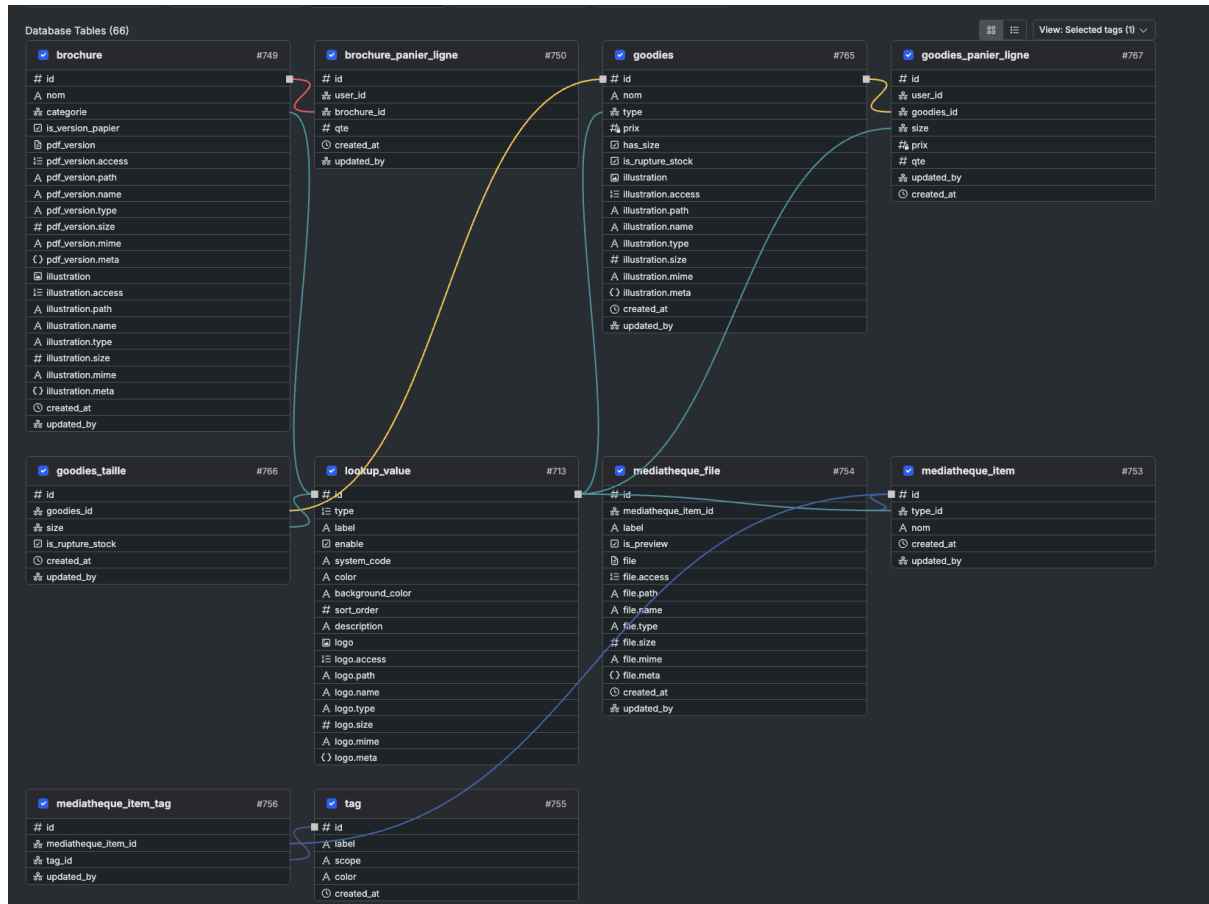
Vue.js (2026). *Vue.js 3 Official Guide*. [En ligne]. Disponible sur : <https://vuejs.org/guide/introduction.html> (Consulté en mai 2026 pour le développement du composant sur mesure).

SOMMAIRE ANNEXES

Annexe 1 : Schéma relationnel de la base de données (Xano).....	34
Annexe 2 : Logique algorithmique Back-end : Fonction create_missing_tags (Xano).....	35
Annexe 3 : Code source du composant sur mesure Input Number (Vue.js).....	36
Annexe 4 : Paramétrage du composant No-Code Active Filter Chips (WeWeb).....	37

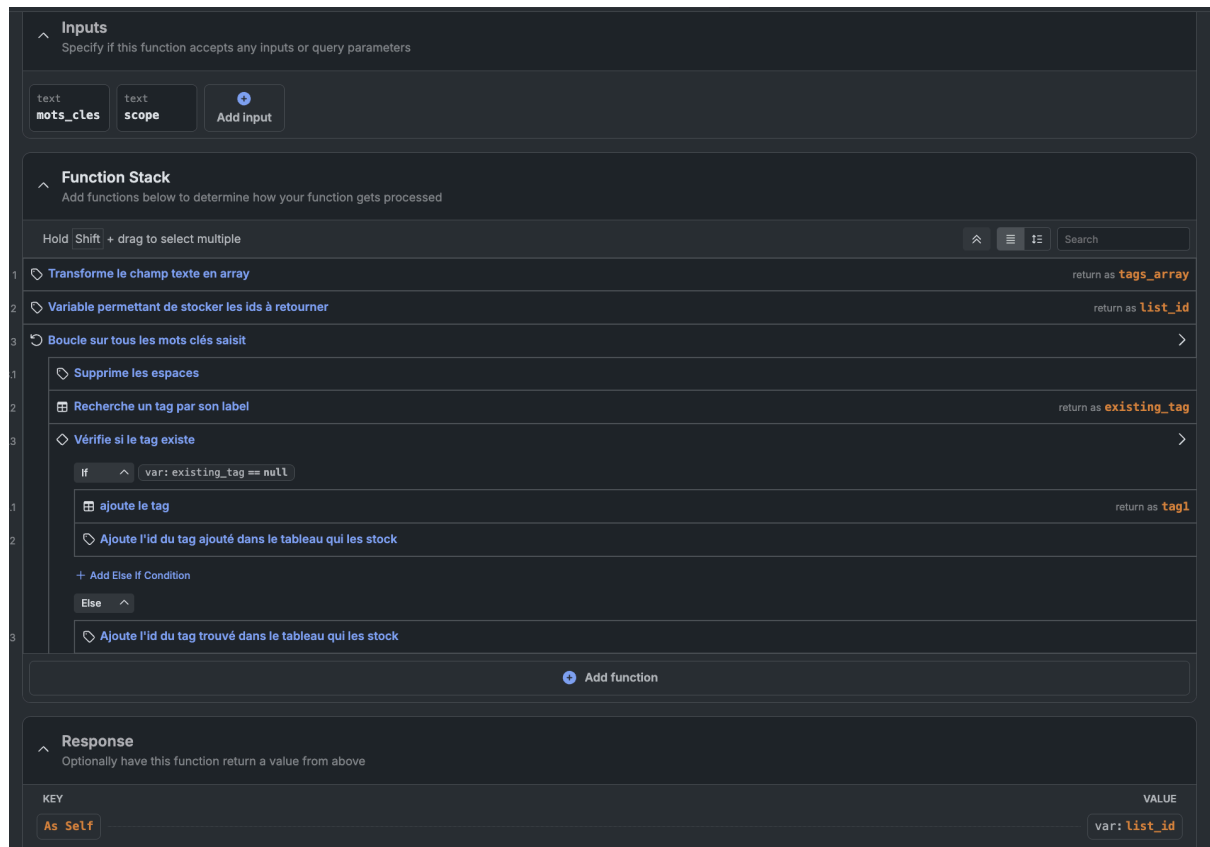
Annexe 1 : Schéma relationnel de la base de données (Xano)

Ce schéma illustre la complexité de l'architecture backend mise en place pour gérer les différentes entités du projet .



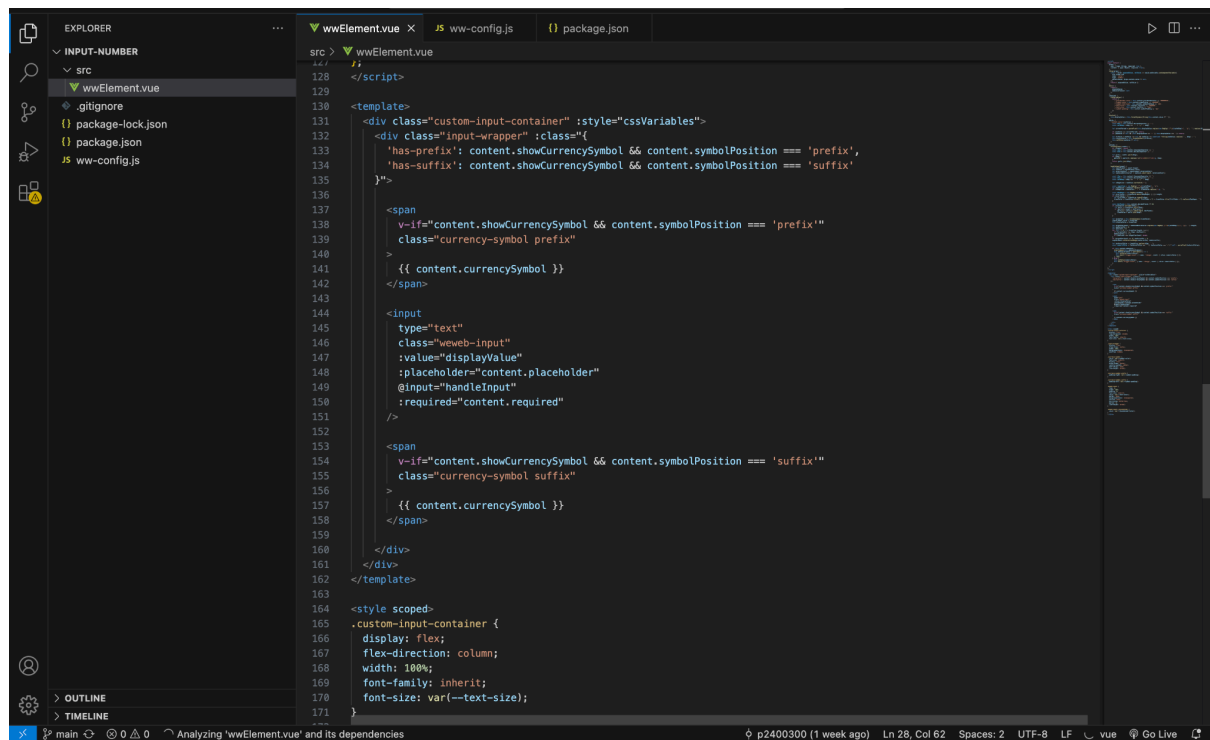
Annexe 2 : Logique algorithmique Back-end : Fonction **create_missing_tags**(Xano)

Capture d'écran de la "Function Stack" illustrant la logique conditionnelle et l'optimisation des requêtes déportées côté serveur (Backend).

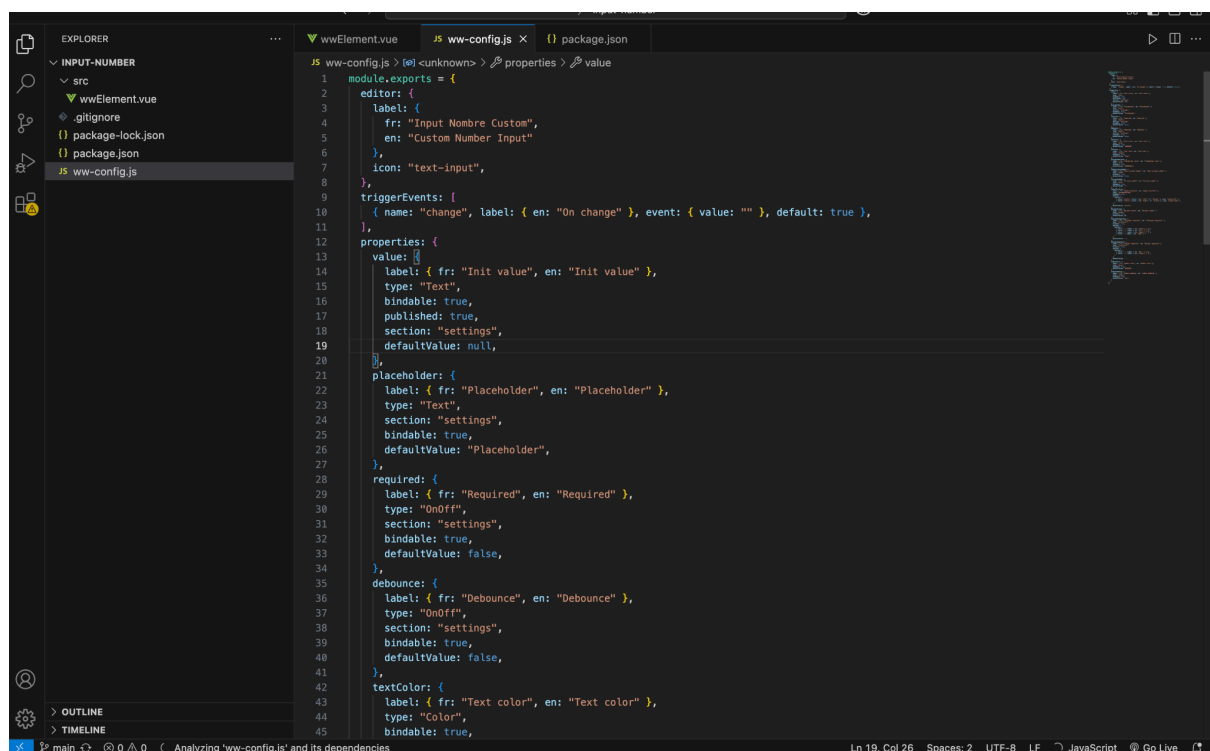


Annexe 3 : Code source du composant sur mesure **Input Number** (Vue.js)

Extrait du fichier **wwElement.vue** illustrant la logique de formatage des nombres et la gestion complexe du curseur, développée spécifiquement pour pallier les limites du Low



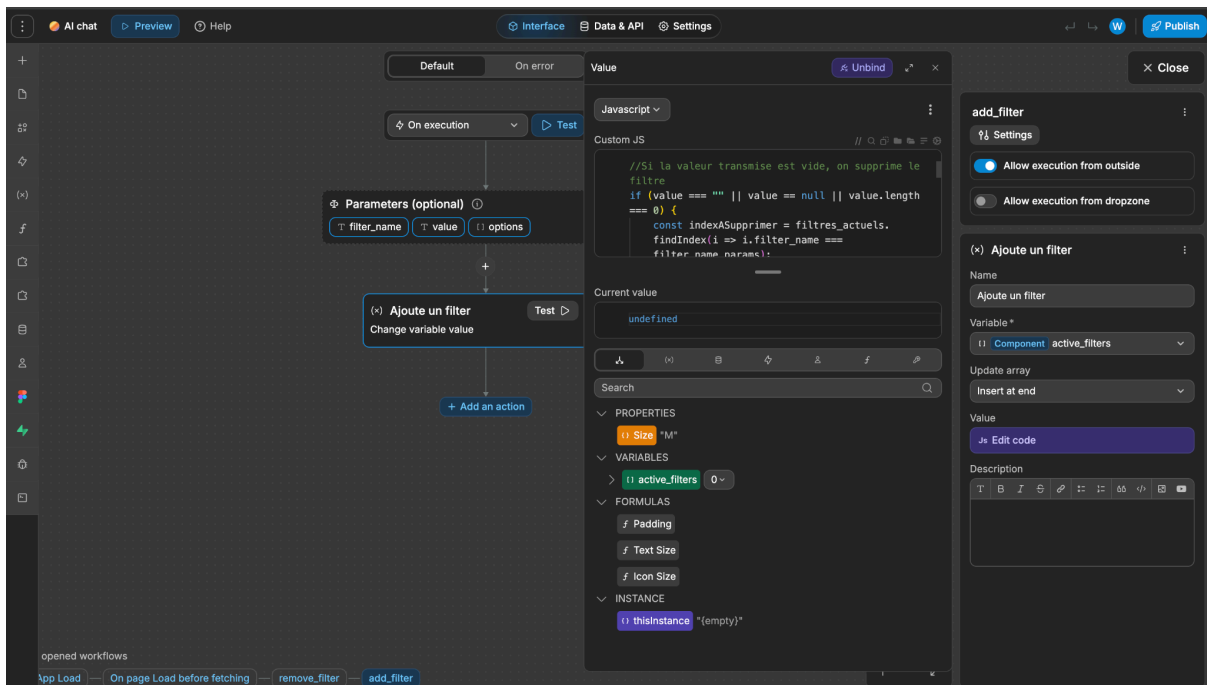
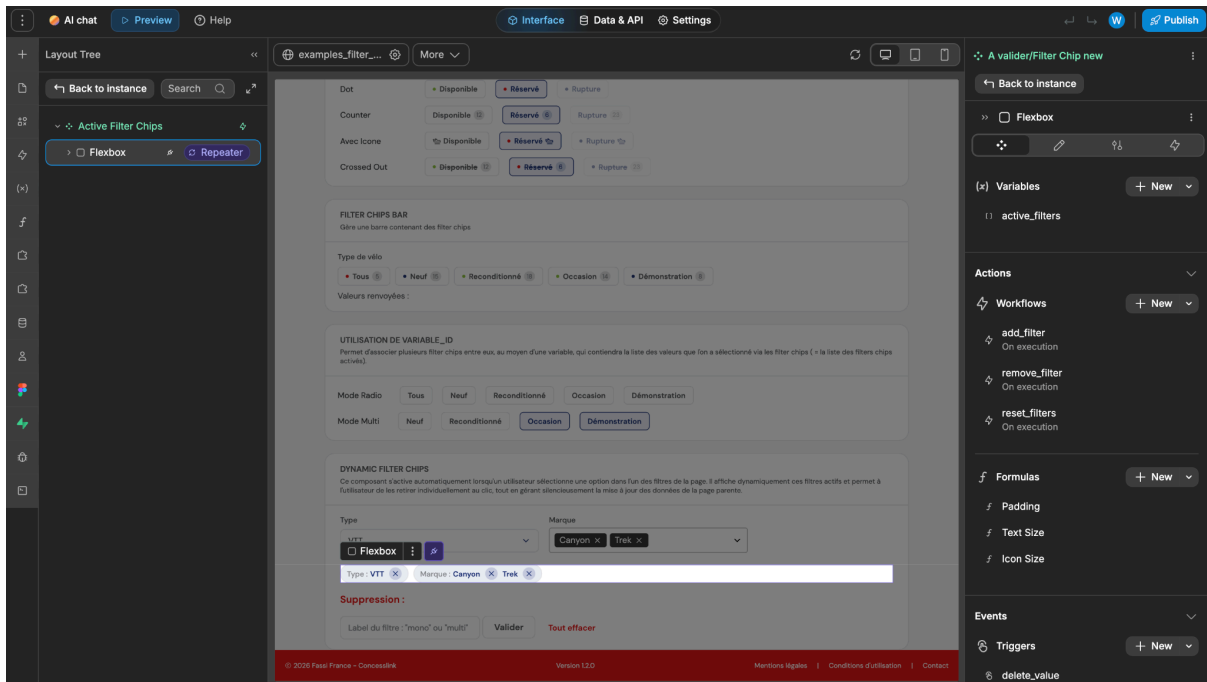
```
128 </script>
129
130 <template>
131   <div class="custom-input-container" :style="cssVariables">
132     <div class="input-wrapper" :class="{
133       'has-prefix': content.showCurrencySymbol && content.symbolPosition === 'prefix',
134       'has-suffix': content.showCurrencySymbol && content.symbolPosition === 'suffix'
135     }">
136
137       <span
138         v-if="content.showCurrencySymbol && content.symbolPosition === 'prefix'"
139         class="currency-symbol prefix"
140       >
141         {{ content.currencySymbol }}
142       </span>
143
144       <input
145         type="text"
146         class="wweb-input"
147         :value="displayValue"
148         :placeholder="content.placeholder"
149         @input="handleInput"
150         :required="content.required"
151       />
152
153       <span
154         v-if="content.showCurrencySymbol && content.symbolPosition === 'suffix'"
155         class="currency-symbol suffix"
156       >
157         {{ content.currencySymbol }}
158       </span>
159     </div>
160   </div>
161 </template>
162
163 <style scoped>
164   .custom-input-container {
165     display: flex;
166     flex-direction: column;
167     width: 100%;
168     font-family: inherit;
169     font-size: var(--text-size);
170   }
171 </style>
```



```
1 module.exports = {
2   editor: {
3     label: {
4       fr: "Input Nombre Custom",
5       en: "Custom Number Input"
6     },
7     icon: "text-input",
8   },
9   triggerEvents: [
10     { name: "change", label: { en: "On change" }, event: { value: "" }, default: true },
11   ],
12   properties: {
13     value: {
14       label: { fr: "Init value", en: "Init value" },
15       type: "Text",
16       bindable: true,
17       published: true,
18       section: "Settings",
19       defaultValue: null,
20     },
21     placeholder: {
22       label: { fr: "Placeholder", en: "Placeholder" },
23       type: "Text",
24       section: "Settings",
25       bindable: true,
26       defaultValue: "Placeholder",
27     },
28     required: {
29       label: { fr: "Required", en: "Required" },
30       type: "OnOff",
31       section: "Settings",
32       bindable: true,
33       defaultValue: false,
34     },
35     debounce: {
36       label: { fr: "Debounce", en: "Debounce" },
37       type: "OnOff",
38       section: "Settings",
39       bindable: true,
40       defaultValue: false,
41     },
42     textColor: {
43       label: { fr: "Text color", en: "Text color" },
44       type: "Color",
45       bindable: true,
```

Annexe 4 : Paramétrage du composant No-Code **Active Filter Chips** (WeWeb)

Capture d'écran de l'éditeur WeWeb dévoilant la logique interne du composant (gestion des variables d'état et émission d'événements).



Value

JavaScript

Custom JS

```
//Si le input est un multiselect
if (is_multiselect) {
  var tab_valeur = Array.isArray(value) ? value : value.split(',');
  var valeurs = []

  for (const valeur of tab_valeur) {
    const valPropre = typeof valeur === 'string' ? valeur.trim() :
    valeur;
    const matchedLabel = options.find(i => i.value === valPropre)?.
    label;
    valeurs.push({ label: matchedLabel, value: valPropre });
  }

  const trouve = filtres_actuels.find(i => i.filter_name ===
  filter_name_params);

  // Si le filtre existe déjà, on remplace ses valeurs
  if (trouve) {
    trouve.valeur.splice(0);
    trouve.valeur.push(...valeurs);
    return;
  }
  // Sinon on crée le nouveau filtre
  else {
    return { "filter_name": filter_name_params, "valeur": valeurs };
  }
}
// Si c'est un mono select
else {
  const singleLabel = options.find(i => i.value === value || i.
```

Current value

undefined

Search

PROPERTIES

Size "M"

VARIABLES

active_filters 0

FORMULAS

Padding

Text Size

Icon Size

INSTANCE

thisinstance "(empty)"

opened workflows

App Load — On page Load before fetching — remove_filter — add_filter